

Home and Away Bear

By

Oscar A. Smalls, Jr

COIN 499, SENIOR PROJECT

SPRING 2008

For

Dr. Lin

April 26, 2008

Contents

Chapter 1. Introduction	1
Develop Project Specification.....	1
Project Objectives	2
Analysis.....	2
Develop Prototype Based on Project Specification	3
Review Project Prototype with Project Advisor	3
Incorporate Feedback into Project Specification	4
Develop Project Timeline	4
Order Required Materials	5
Chapter 2. Hardware	6
The Bear.....	6
The Boards.....	10
The Modules	14
Infrared Communication	15
MP3 Player.....	24
Bluetooth Communication	28
System Status.....	34
Chapter 3. Software	36
Software Requirements.....	36
Software Design.....	36
Chapter 4. Results.....	43
Chapter 5. Conclusion.....	44
Summary.....	44
Areas of Enhancement.....	45
Chapter 6. Appendix	46
Microsoft Agent “Peedy” Actions	46
Music in “My Museum”	48
Backgrounds in “My Museum”	48
Button GIFs in “My Museum”	49
Sample MP3 of NASA Moon Landing.....	49
Code: IRSEND.bs2	50
Code: IRRECEIVE.bs2.....	53
Code: Bluetooth.cs.....	65
Code: CountGame.cs	70
Code: IdentifyGame.cs	72
Code: Image.cs.....	74
Code: ImageGame.cs.....	76
Code: MyMuseum.cs	80
Code: Selection.cs.....	95
Chapter 7. References	97

Chapter 1.

Introduction

Develop Project Specification

The goal of this project is to create a device that can be used with a home computer and at a museum or similar type of attraction equipped to communicate with the device. The device will be in the form of a bear to appeal to children. When the device is used at home with software that can communicate with the device, it will wirelessly provide a unique computer experience where the device and the computer speak to each other in response to the user's choices on the computer screen. If the device is taken to an attraction with exhibits that can communicate with it, the device will tell the user about the exhibit in the attraction when the device passes by that exhibit.

This device would be entertaining for the home computer user **who** would like to add an external interactive element to their computer experience. Attraction owners would benefit from this device because it adds a new level of experience to their attraction. Children would also benefit from using this device in attractions because the bear would be a comfortable way to open a child's mind to important exhibits in an art galley or museum.

Parts of this project have been done before by others. Pal Mickey is a device used at Walt Disney World's theme parks to inform guests about attractions, but it cannot be used with a home computer. Museums have similar devices, but not in the shape of a

toy. Interactive Pooh by Disney and Microsoft's Actimates Barney are two toys used at home that could connect to a computer to provide an interactive experience, but neither is made by their respective companies anymore. My Home and Away Bear would be the only device that I am aware of that can be used with a home computer and at attractions away from home.

I chose this project because I have always had an interest in electronics, robotics, computers, and animation.

Project Objectives

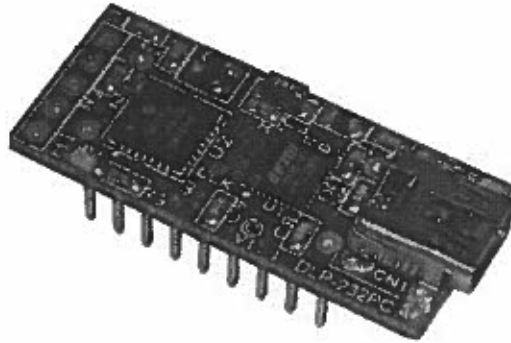
I hope to accomplish the creation of a device that can be used without a hardwired connection to a computer, and I hope to learn infrared technology, electronic circuitry, Bluetooth technology, and DirectX programming. This project will also use the following languages in which I am familiar: C# and Basic Stamp.

Analysis

The first step in completing this project required getting approval from my project advisor, Dr. Lin. I explained my interest in developing this device and he challenged me to create a proof-of-concept in two weeks which included a circuit board with an infrared transmitter, different circuit board with an infrared receiver, and a way for the infrared receiver to communicate with a PC via a USB connection. Since I was familiar with Parallax BASIC Stamp microcontroller circuit boards, I referred to my IR Remote for the Boe-Bot book to create an infrared transmitter and an infrared receiver.

Develop Prototype Based on Project Specification

In order to allow the infrared receiver to communicate with a personal computer, I purchased a DLP-232PC microcontroller which can listen for digital circuit signals and send those signals to a computer when wired to the computer's USB port.



DLP-232PC

While working on the prototype, I learned more about infrared circuits, and particularly, how to make the infrared signal work at a distance greater than the circuits described in the IR Remote for the Boe-Bot book. However, these long range infrared circuits appeared to require an encoder chip on the infrared transmitter and a decoder chip on the infrared receiver.

Review Project Prototype with Project Advisor

I was able to complete the prototype in two weeks and have it reviewed by my project advisor. Dr. Lin mentioned that I should not try to figure out how to integrate encoder and decoder chips into my project, but I should have the program running on the microcontroller do the encoding and decoding since this is a computer science project

and not an engineering project. It was also determined that the DLP-232PC microcontroller was not a necessary component in my senior project.

Incorporate Feedback into Project Specification

With encoders, decoders, and the DLP-232PC removed from the project, I focused on creating simpler IR transmitter and IR receiver circuits which depend on the BASIC Stamp microcontroller for encoding and decoding. Detailed implementation steps were also added to the project specification.

The Home and Away Bear project consists of four major components: infrared communication to allow the bear to notify the user of exhibits in a museum, an MP3 player in the bear to permit the bear to say something relevant about a museum exhibit or computer program in real-time, a Bluetooth module in the bear and another Bluetooth module in the computer in order for the bear to communicate wirelessly with the computer, and a graphical computer program written in C# that can demonstrate the bear responding to the user's choices in the computer program.

Develop Project Timeline

In fall 2007, the original timeline for this project allowed the project to be developed over three semesters. However, since I have determined that I am eligible to graduate at the end of the spring 2008 semester, this three semester project has been compressed into two semesters and the original timeline is no longer valid.

In general, the timeline for this project has gone along the lines of:

September 2007	Configure IR Transmitters (there are two of them)
October 2007	Configure IR Receiver
November 2007	Configure MP3 Playback
January 2008	Configure Bluetooth Module on Bear
February 2008	Configure Bluetooth Module on PC
March 2008	Develop Graphical Program for PC
April 2008	Testing and Documentation

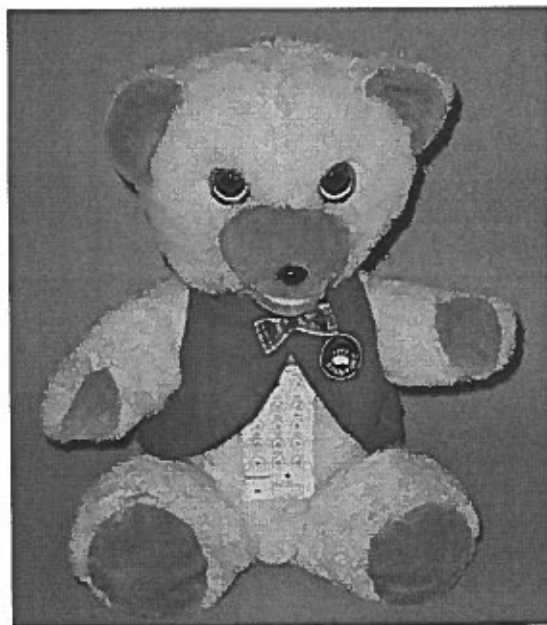
Order Required Materials

After the detailed implementation steps were defined, the hardware required to implement the project was ordered. Chapter 2 covers the hardware implemented and a component list is provided in the appendix.

Chapter 2.

Hardware

The Bear

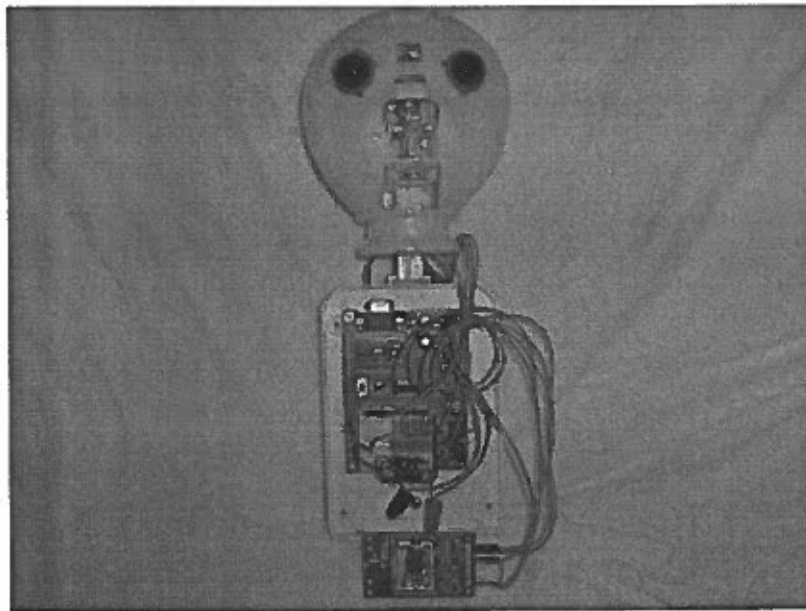


The bear used in this project was originally a speakerphone. K. C. Bearifone II was manufactured by TeleConcepts Inc. in 1986 and can be purchased today at http://www.customphones.zoovy.com/product/44-KCBEAR/KC_Bearifone_Talking_Bear_Telephone.html. He does not come with a handset, but his eyes and mouth move when you are listening to someone over his speakerphone. He also had optional accessory with a microphone that could plug into him so that you can make his eyes and mouth move too. The bear pictured above is from the website listed above. I took the speakerphone out of my bear years ago in

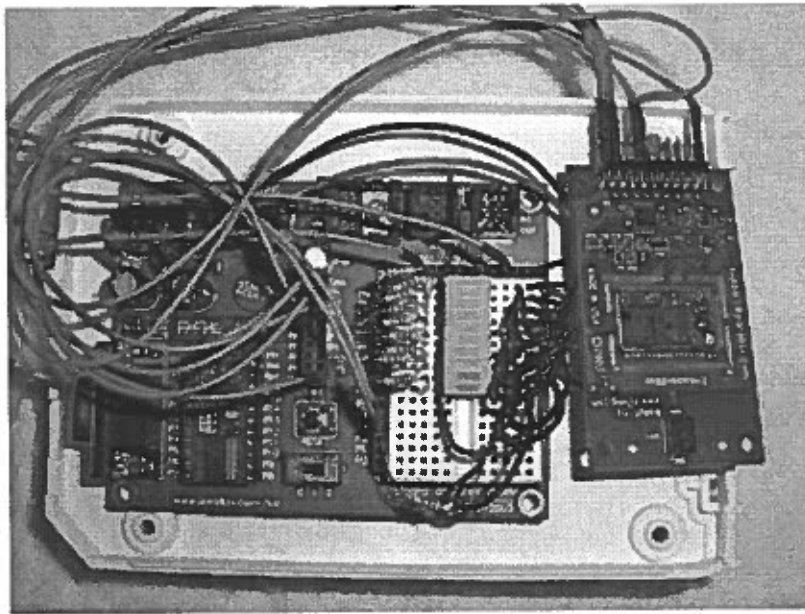
order to figure out how to control the bear's eyes and mouth with computer software.

The bear's mechanical movement is not part of this project, but I do plan to incorporate movement in the bear some day in order to study the "uncanny valley" effect.

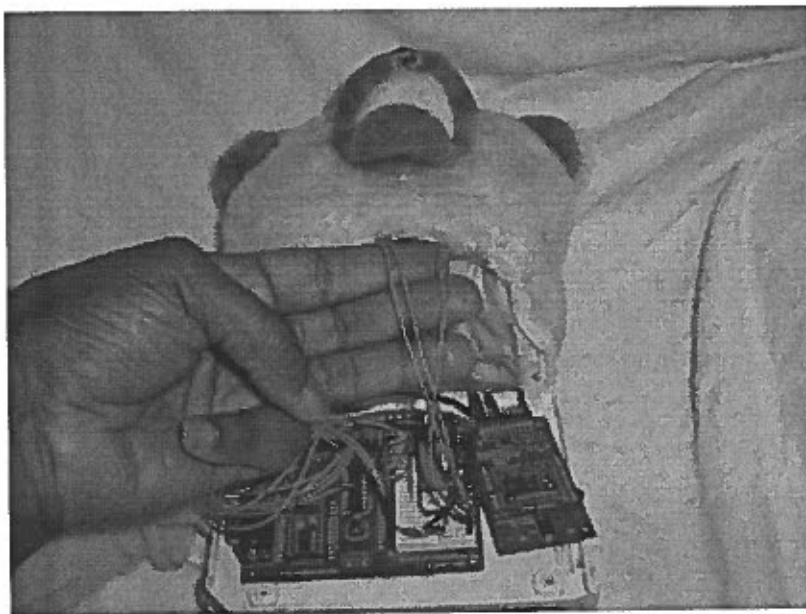
The bear ended up being the perfect device for this project. It is very lightweight and weighs no more than Disney's Interactive Pooh or Actimates' Interactive Barney. The shell that was used for the speakerphone now holds the larger main circuit board. The smaller board pictured is the bear's Bluetooth module.



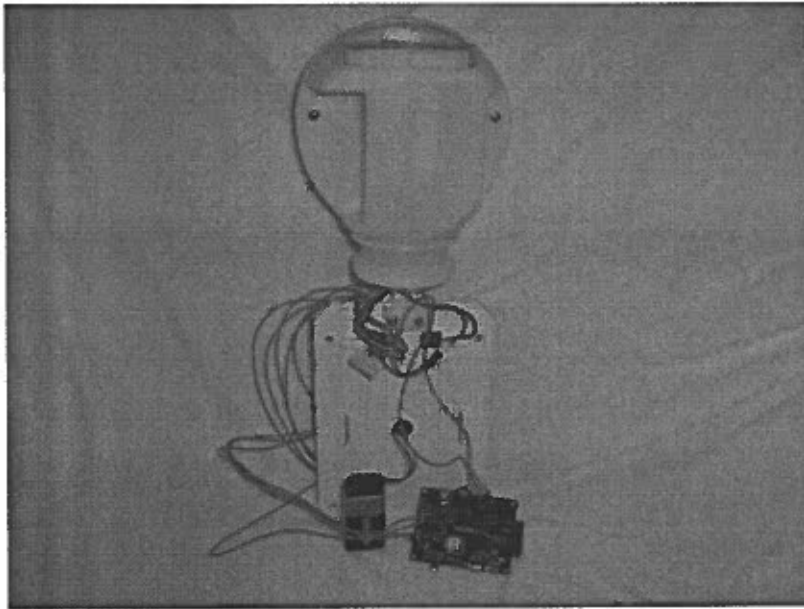
Below is a close-up of the main circuit board and the Bluetooth Module.



The infrared detector is mounted under the nose of the bear and is not connected to the main board in the previous picture because it is attached to the face of the bear. The picture below shows the three wires going to the infrared detector inside the head.



The back of the bear has a shell that covers the bear's 9V battery and MP3 Player. The thing sticking a little bit out of the right side of the MP3 Player is a one gigabyte SD card. The bear's speaker is at the top of his head.

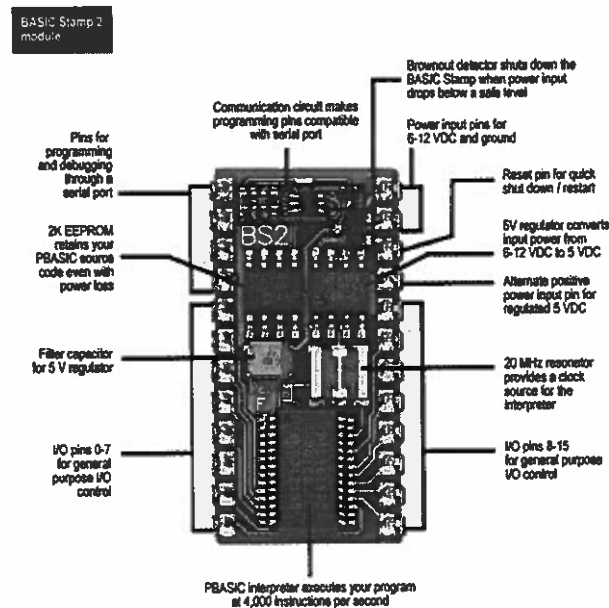


And this is what the finished product looks like...



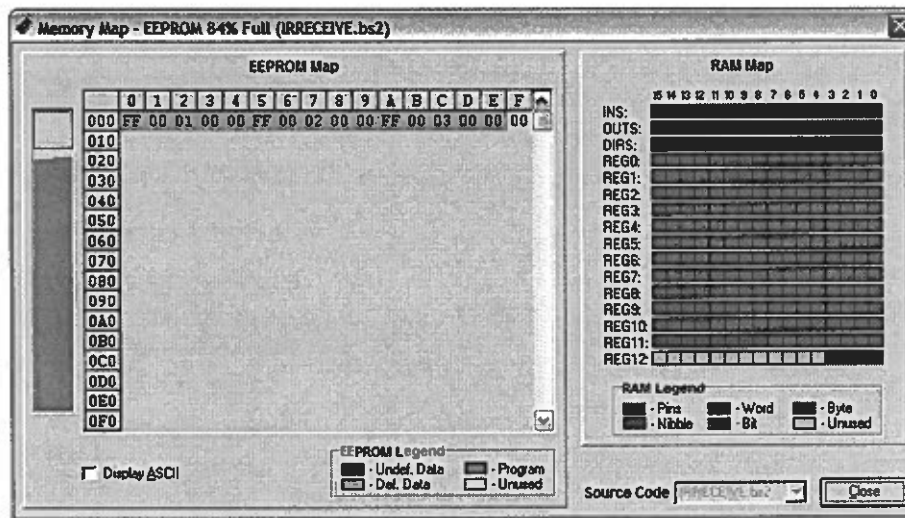
The Boards

The bear's main circuit board and the circuit board of the IR transmitters have a microcontroller on them called BASIC Stamp 2. This microcontroller has a small BASIC interpreter that operates at 20 kHz and can handle 4,000 instructions per second. It has a RAM size of 32 bytes, of which 6 are dedicated to input/output operations and 26 are available for variables. There are 2K bytes of EEPROM (Electrically Erasable Programmable Read-Only Memory) storage available programs (up to 500 instructions) and data. Sixteen I/O pins are available. However, the processor is single-threaded. Below is a picture of the BASIC Stamp 2 microcontroller:

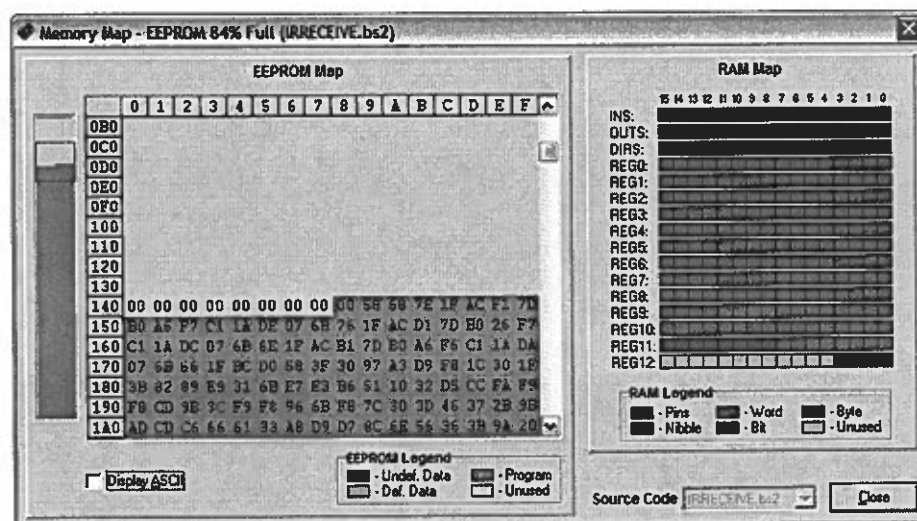


The memory map of the program stored in the EEPROM of BASIC Stamp on the main circuit board is shown on the left below. At compile time, the program stores three

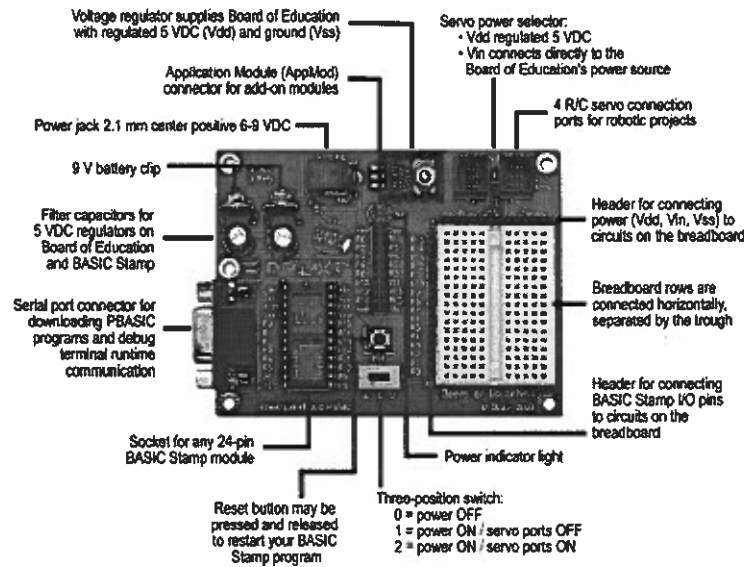
pieces of data in the EEPROM's memory: FF 00 01 00 00, FF 00 02 00 00, and FF 00 03 00 00. Data is stored in EEPROM starting from lowest memory address available.



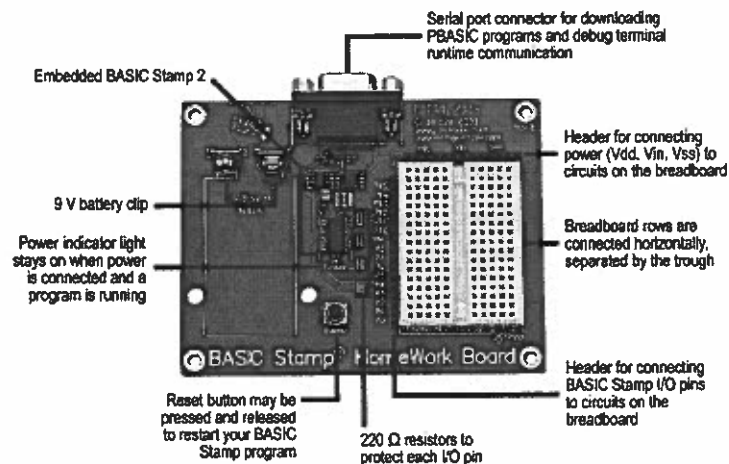
The memory map shown below is of the same program, just a different memory address. Programs are stored starting from the highest memory address. Note that the memory is 84% full. On the right is the RAM Map. The 6 bytes dedicated to I/O are in INS, OUT, and DIRS. I have less than two bytes of unused RAM because I have a lot of variables.



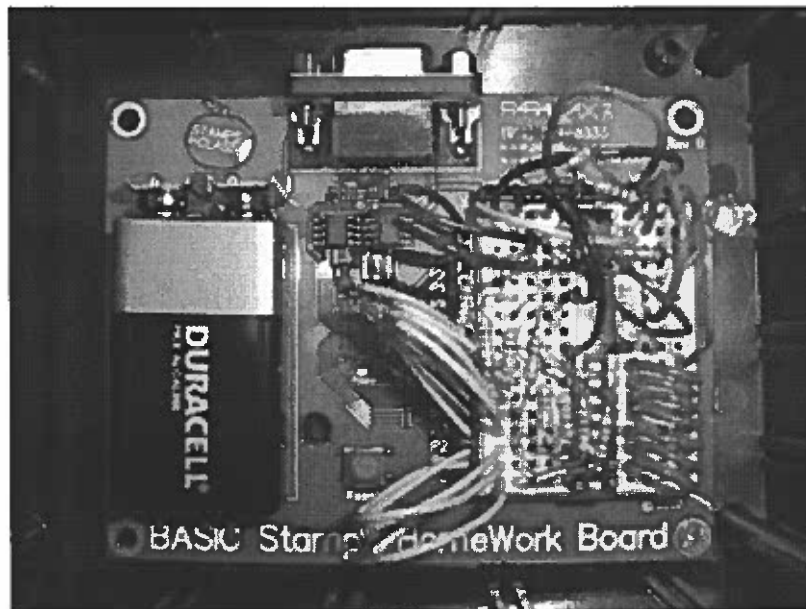
“The BASIC Stamp is a very small computer. This very small computer plugs into the Board of Education, which is called a carrier board. The Board of Education makes it easy to connect a power supply and serial cable to the BASIC Stamp” (“What’s”, 13).



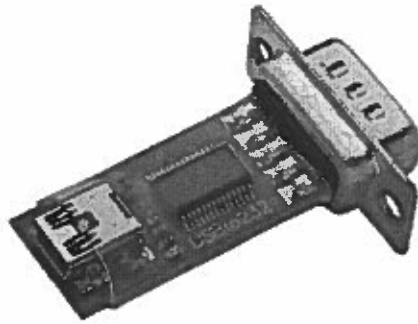
“The BASIC Stamp HomeWork Board is like a Board of Education with the BASIC Stamp 2 built-in” (“What’s”, 14).



This project uses one Board of Education for the bear's main circuit board and two HomeWork boards for the IR Transmitters. One IR Transmitter is shown below:



Since all of these carrier boards have serial ports and my computer only has USB ports, I need to use the Parallax USBto232 adapter shown below to connect to the carrier board in order to reprogram the BASIC Stamp 2 microcontroller.



Parallax now makes USB carrier boards for the BASIC Stamp 2 microcontroller. One last thing to note about carrier boards: It is best to have a couple extra carrier boards so that you can test out new circuits on them before adding the new circuit to your main circuit board.

The Modules

There are two sides to the Home and Away Bear: a “Home” side and an “Away” side. The “Home” side occurs when the bear is on and near the PC, the PC’s Bluetooth module is plugged in, and the user opens up the program “My Museum”, described later in this document. The bear will notice that the program has been opened and say “Welcome to My Museum.” The user can make selections in “My Museum” that will make the bear say something. The “Away” side occurs when the bear is first turned on and the user does not have the “My Museum” program open. When the user brings the

bear within six feet of an IR transmitter that the bear can see and understand, the bear will say something about that particular IR transmitter.

The “Home” side makes use of the Bluetooth Communications, the MP3 player, System Status, and the Software Development modules. The “Away” side makes use of the Infrared Communications, the MP3 player, and System Status modules. Since I had already started working with Infrared Communications when working on the proof-of-concept prototype, I continued working on all of the “Away” side modules before approaching the remaining Software Development module on the “Home” side.

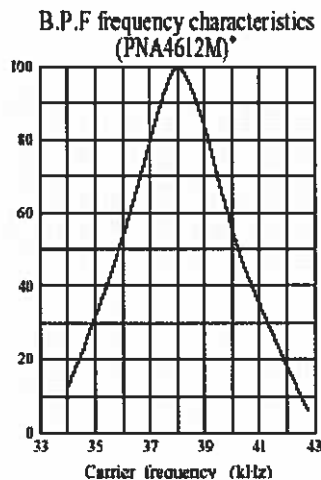
With the exception of the system status module, each of the hardware design modules and the software development module had its own challenges that I had to understand and overcome. The rest of this paper will cover what I had to learn to in order to implement each module.

Infrared Communication

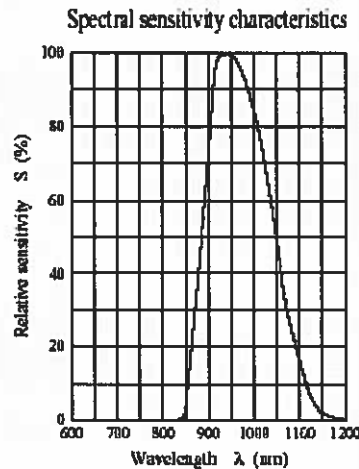
While during research on infrared communication, I was focused on trying to figure out how to create a long range infrared communication system. The best device that I could find that would work well in delivering reliable long range infrared communication was the Panasonic 38 kHz photodetector – PNA4612M. The datasheet by Panasonic states that it should work at distance of 11 meters or more. A photodetector is a device exactly like the one that is built into your television set that detects signals when you press buttons on your remote control. The 38 kHz frequency is a “carrier signal.” “The IR remote uses a . . . ‘carrier signal’ to transmit the pulse durations from the remote to the

IR detector” (“IR”, 5). Each key that you press on your television remote control sends different pulse durations to tell the television what you want to do. The same frequency has to be both the IR detector and the IR remote. This understanding between the IR detector and the IR remote is called a communication protocol. “A communication protocol is a set of rules for devices that have to exchange electronic messages. Protocols tend to have rules for voltages, the amount of time signals last, carrier signal frequencies and/or wavelengths, and much more. When two or more devices follow the rules of a given protocol, they should be able to communicate and exchange information” (“IR”, 5).

The datasheet for the Panasonic PNA4612M photodetector has the following chart showing which frequency works best for it. As expected, the frequency is 38 kHz.



Panasonic’s PNA4612M datasheet also includes the following chart:



This chart notes that the PNA4612 works best at wavelength 940nm. To get a better idea of what 940nm means, consider the following chart from Parallax's Robotics with the Boe-Bot book. All values are in nanometers (nm):

Colors and Approximate Wavelengths			
Color	Wavelength	Color	Wavelength
Violet	400	Red	780
Blue	470	Near infrared	800-1000
Green	565	Infrared	1000-2000
Yellow	590	Far infrared	2000-10,000
Orange	630		

“Infra means below, so Infra-red is light (or electromagnetic radiation) that has lower frequency, or longer wavelength than red light . . . Night-vision goggles and IR temperature sensing use far infrared wavelengths” (“Robotics”, 235). “We’ll be using a detector that has peak sensitivity at a wavelength of 940nm, so we’ll use IR LEDs that emit at 940nm” (“97.497: RoboFlag”,13). “The Lumex OED-EL-1L2 infrared LED is a high-power, wide-angle LED that transmits at our desired wavelength of 940nm . . . and has a 60-degree beam width” (“97.497: RoboFlag”,42). “Selecting the best parts for the

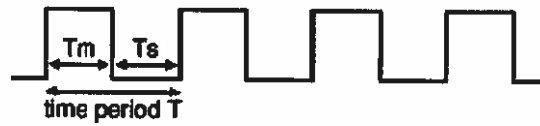
IR beacon is important. If you're using an IR LED that doesn't transmit at 940nm, for example, the PNA4612M receiver module won't work as well" ("97.497: RoboFlag", 65).

Before going any further, let's get a better understanding of what 38 kHz means.

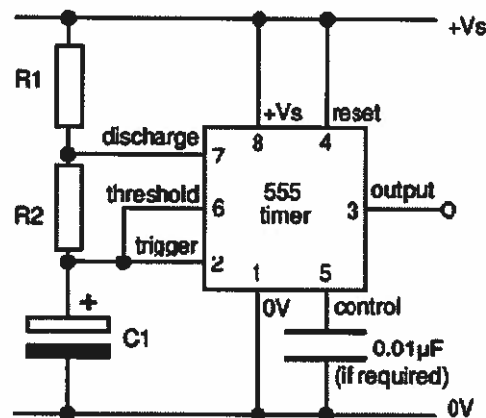
"Just about all of the electronic beeps you hear during your daily routine are made microcontrollers connected to speakers. The microcontroller creates these beeps by sending rapid high/low signals to various types of speakers. The rate of these high/low signals is called the frequency, and it determines the tone or pitch of the beep. Each time a high/low repeats itself, it is called a cycle. You will often see the number of cycles per second referred to as Hertz, and it is abbreviated Hz. For example, one of the most common frequencies for the beeps that help machines get your attention is 2 kHz. That means that the high/low signals repeat at 2000 times per second" ("What's", 215). So, that also means that we need to flash the IR transmitter's OED-EL-1L2 infrared LED 38,000 times per second to just establish a "carrier signal" to the IR receiver's Panasonic PNA4612M photodetector.

IR Transmitter

One way to flash the IR Transmitter's LED with a frequency of 38 kHz is use an astable circuit. "An astable circuit produces a 'square wave', this is a digital waveform with sharp transitions between low (0V) and high (+Vs) . . . The circuit is called an astable because it is not stable in any state: the output is continually changing between 'low' and 'high'." ("555", 28).



“The time period (T) of the square wave is the time for one complete cycle, but is usually better to consider frequency (f) which is the number of cycles per second” (“555”, 29). In order to get have the IR transmitter produce square waves like above for a 38 kHz frequency, we need to implement the astable circuit shown below. We just need to find out values for resistors R1, R2, and the capacitor C1.



$$T = 0.7 \times (R1 + 2R2) \times C1$$

$$f = 1.4 / [(R1 + 2R2) \times C1]$$

T = time period in seconds

f = frequency in hertz

R1 = resistance in ohms

R2 = resistance in ohms

C1 = capacitance in farads

The time period can be split into two parts: $T = T_m + T_s$

Mark time: $T_m = 0.7 \times (R_1 + R_2) \times C_1$

Space time: $T_s = 0.7 \times R_2 \times C_1$

For a standard astable circuit T_m cannot be less than T_s .

Choosing R_1 , R_2 , and C_1 :

- Choose C_1 to suit the frequency range you require (see table at <http://www.kpsec.freeuk.com/555timer.htm>).
- Choose R_2 to give the frequency (f) you require. Assume that R_1 is much smaller than R_2 (so that T_m and T_s are almost equal), then you can use: $R_2 = 0.7 / (f \times C_1)$.
- Choose R_1 to be about a tenth of R_2 (1kiloohm minimum) unless you want the mark time T_m to be significantly longer than the space time T_s .
- If you wish to use a variable resistor, it is best to make it R_2 .

$C_1 = 0.01$ microfarads

$R_2 = 0.7 / (f \times C_1) = 0.7 / (38,000 \text{ Hz} \times 0.00000001 \text{ Farads}) = 1842 \text{ ohms}$

$R_1 = (1/10) \times 1842 \text{ ohms} = 184 \text{ ohms}$

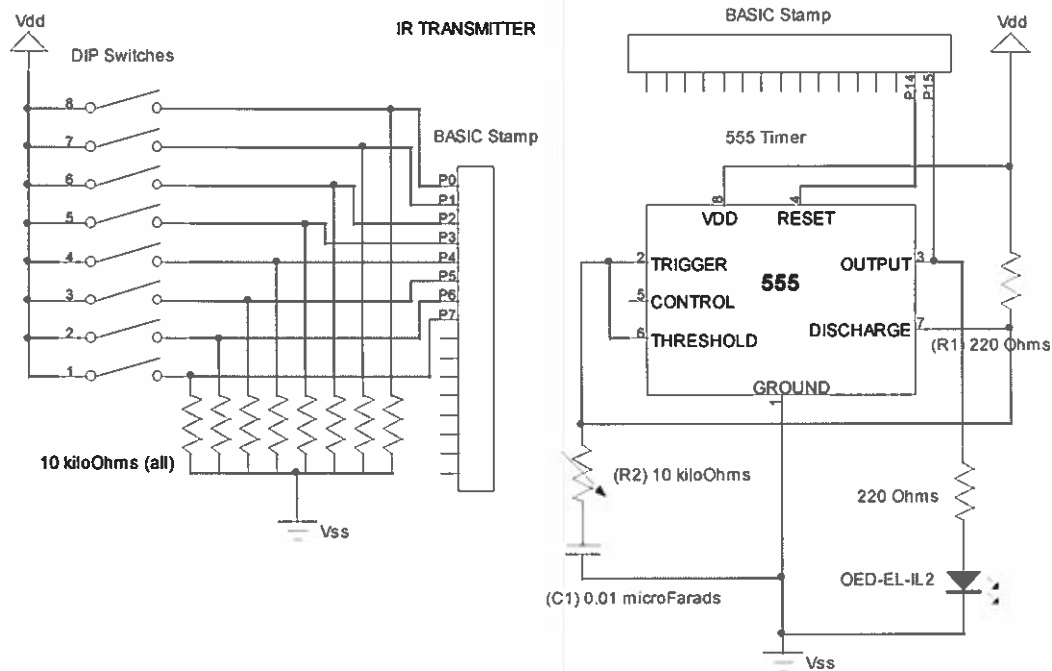
We will let R_1 be 220 ohms, which is close to 184 ohms and let R_2 be a variable resistor (potentiometer) that goes from zero ohms to 10,000 ohms. Now when we solve $f = 1.4 / [(R_1 + 2R_2) \times C_1]$ for R_2 , we get $R_2 = 1732 \text{ ohms}$

$T_m = 0.7 \times (R_1 + R_2) \times C_1 = 0.7 \times (220 \text{ ohms} + 1732 \text{ ohms}) \times 0.00000001 \text{ Farads} = 0.000013202 \text{ seconds}$

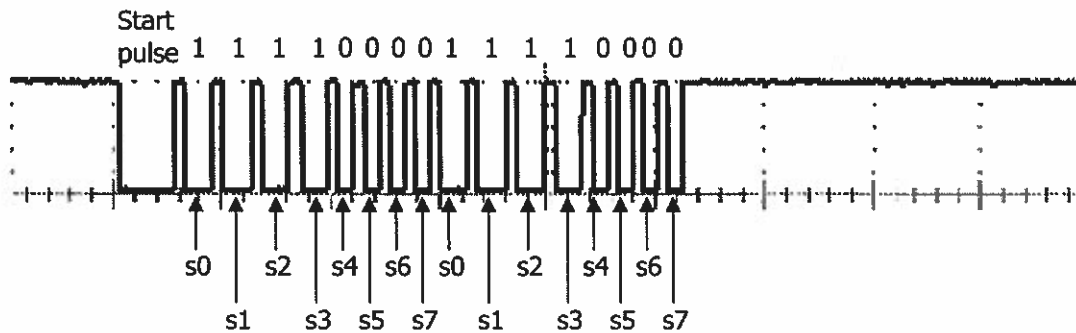
$T_s = 0.7 \times R_2 \times C_1 = 0.7 \times 1732 \text{ ohms} \times 0.00000001 \text{ Farads} = 0.000012124 \text{ seconds}$

T_m and T_s are almost equal and T_m is greater than T_s , so this is a standard astable circuit.

The circuit on the right was built from letting $R1 = 220\ \text{ohms}$, $R2 = 10\ \text{kilo ohm}$ variable resistor and $C1 = 0.01\ \text{microfarads}$. We are letting the BASIC Stamp microcontroller send information on the 38 kHz “carrier signal” that the astable 555 timer circuit generated. The information that the BASIC Stamp is sending comes from the circuit on the left. The BASIC Stamp is reading eight DIP switches corresponding to the specific id of this particular IR transmitter. These ids can range from 00000001 bytes to 11111111 bytes, allowing 255 addressable IR transmitters. An IR transmitter id of 00000000 is not recognized.



When the BASIC Stamp reads the id of the IR transmitter, it sends the id out as pulses on the 38 kHz signal to the OED-EL-IL2 infrared LED, just like how your television remote sends signals to the television.

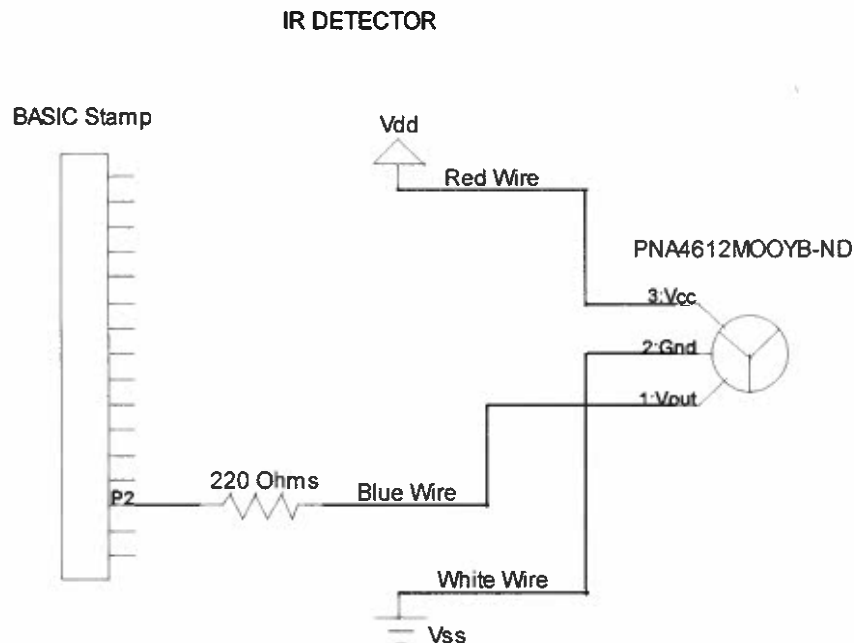


The figure above shows a timing diagram for IR transmitter with the id of 00001111.

The first pulse is the start pulse, which lasts for 2.4 ms. The pulses are then transmitted in LSB-first order. This stands for least significant bit first, meaning the first data pulse is bit-0, the next data pulse is bit-1, and so on. The transmitter's id is sent twice for error checking by the IR receiver. After sending the id is sent a second time, the transmitter waits for about 23 ms. It then sends another start pulse and sends the id twice again. Binary-1 is represented by sending a pulse of 1.2 ms and binary-0 pulses for the duration of 0.6 ms. Note that the signal is high when there is no pulse and the signal goes low when there are pulses transmitted. "This communication scheme is called pulse width modulation (PWM), because when graphed against time, the IR detector's high/low signals form pulses of different widths that correspond to their durations" ("IR",164).

The code for the IR transmitter is called IRSEND.bs2 and can be found in the Appendix. Pulse width is calculated by looking at the bit of the DIP switch and multiplying it by 300 and then adding 300 to the answer. If the DIP switch had a value of 0, the program would pulse 300. If the DIP switch had a value of 1, the program would pulse 600. A pulse of 300 would last for 0.6 ms and a pulse of 600 would last for 1.2 ms.

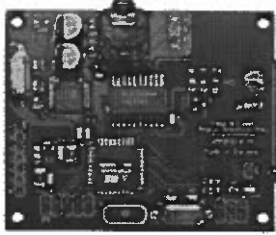
IR Receiver



The circuit for the IR receiver is shown above and resides on the main circuit board located in the bear. The PNA4612M infrared detector is located under the nose of the bear. It sends signals that it detects to the BASIC Stamp so that the BASIC Stamp code can decide what to do with the signal. When the BASIC Stamp detects two IR transmitter ids sent correctly one after the other, it calls the MP3 module to play the MP3 file associated with that IR transmitter. The BASIC Stamp also displays the binary number of the IR transmitter on the LED display on the main circuit board.

The code that the IR receiver uses is part of the IRRECEIVE.bs2 code found in the Appendix.

MP3 Player



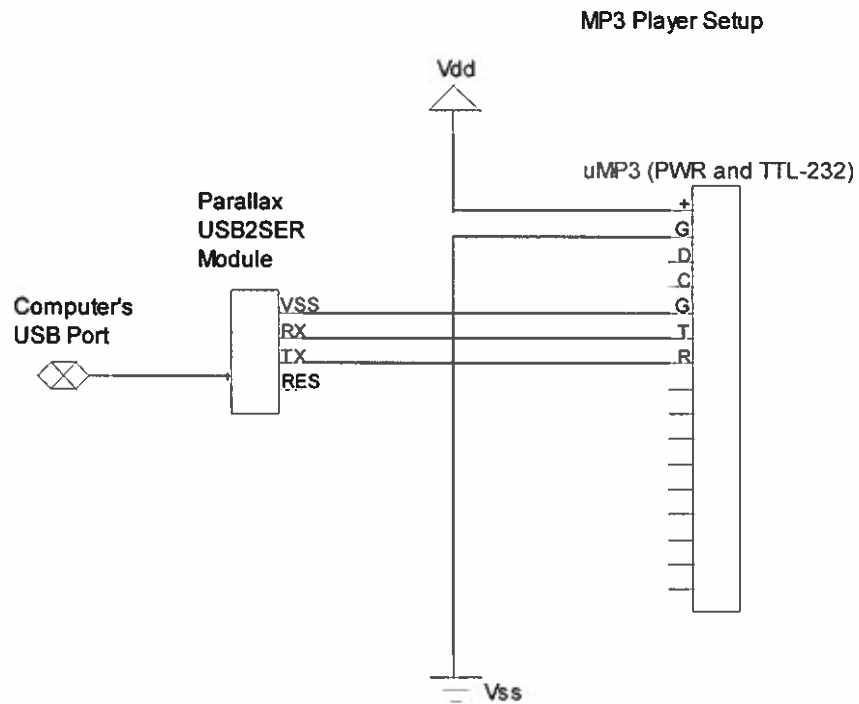
The MP3 player is used in this project to play MP3 files corresponding to a specific IR transmitter, a specific phrase the bear needs to say in response to the “My Museum” software program, or just to make sure that the bear has finished initialization and is ready for use. This MP3 player is made by Rogue Robotics and is called the uMP3 Playback Module.

Jon Williams article “Sounds of the Season” in volume 6 of Parallax’s The Nuts and Volts of BASIC Stamps series helped me get started in using the uMP3 by connecting it up to HyperTerminal to do some basic configuration and checking to make sure it was working.

“Before connecting the uMP3 player to the BASIC Stamp we need [to] configure it with a terminal program. Once we’ve adjusted the settings to our liking . . . we don’t have to worry about making them again unless we update the firmware” (Williams, 266). In order to connect the uMP3 to a personal computer, you need to use a device such as the Parallax USB2SER module shown below.

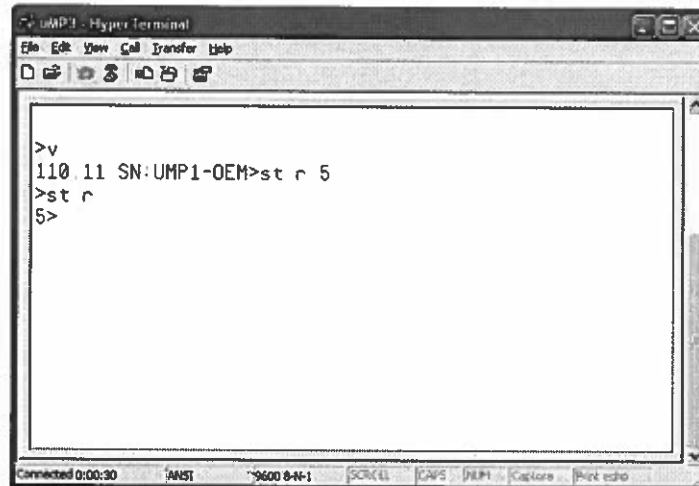


The schematic on how to hook up the board is shown below. Note the two ‘G’ connections on the uMP3.



“Using a terminal program we can check the firmware and update the settings to make things operate smoothly with the BASIC Stamp . . . Out of the box, the uMP3 is configured for 9600 baud so that’s what the terminal was set for. At power up the uMP3

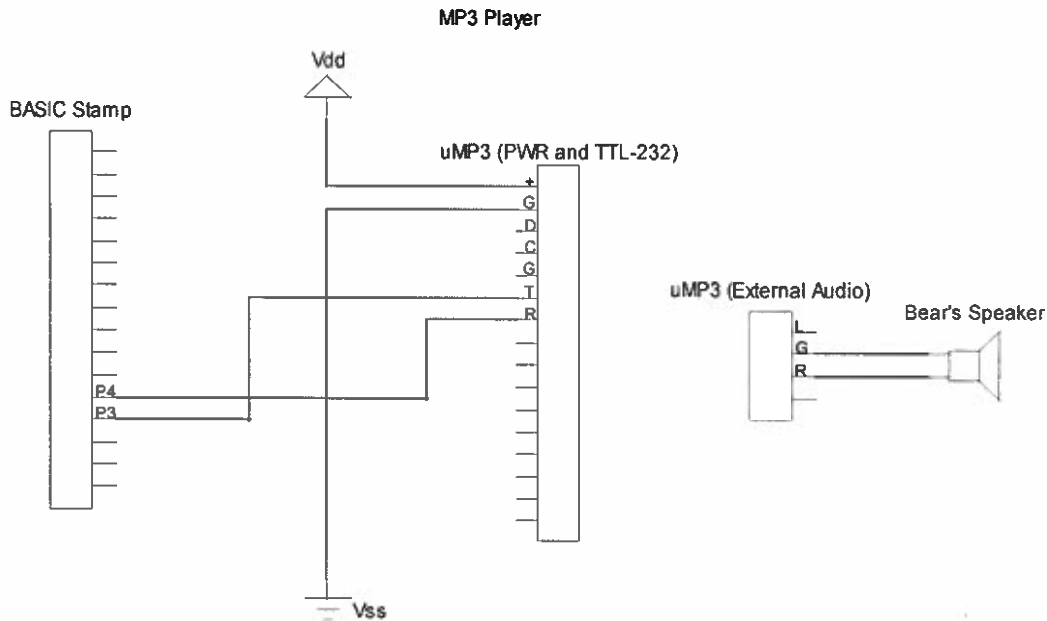
scans the connected media (SD card) and when ready it will transmit a ">" to the terminal" (Williams, 266).



```
>v
110.11 SN:UMP1-OEM>st r 5
>st r
5>
```

"We can check the firmware version with the "v" (no quotes) command. Note that all uMP3 commands are terminated with a carriage return, and we especially have to remember this for our BASIC Stamp program . . . Before connecting to the BASIC Stamp we want to change the prompt delay to five milliseconds . . . Change the prompt delay with "st r 5" and then confirm the setting entering "st r" without a delay time. The current delay will be reported before the prompt character (it should be a 5)" (Williams, 266).

The next step in Williams' article was to connect the uMP3 to the BASIC Stamp circuit board. Since I did not have any experience in working with TTL serial connections, I mistook one of Williams' instructions and hook the uMP3 up to the wrong ground and destroyed the board. Below is a schematic of how to hook the board up properly:



Since the uMP3 does not come with any pins or breadboard sockets to securely connect wires, I had to buy a soldering iron and solder pins where I needed them to be. I am pretty sure that my soldering destroyed something on another uMP3 board, causing me to buy a third one.

The uMP3 does a good job of playing MP3 files off my 1GB SD card.

To create MP3 files for an IR transmitter:

- Record the MP3 file with a personal computer; file size is restricted to free space on the SD card
- Set the IR transmitter's dip switch settings s0 through s7 to an available transmitter id
- Rename the MP3 file to 0xxxxxxx.MP3 where xxxxxxxx is the transmitter id
- Insert SD card with file into MP3 module

When creating files for the “My Museum” software program, precede the eight bit file name with a one as in 1xxxxxxx.MP3. Doing this allows lets the bear to say up to 255 different phrases for museum exhibits and 256 phases for the “My Museum” software program. Note that transmitter id 00000000 is not allowed.

The uMP3 is connected directly to the bear’s speaker located at the top of the bear’s head. The position of the speaker aids in directing what the bear is saying to the person holding the bear and not to other people nearby.

If a user triggers the bear to say something and the bear is still talking while the user triggers the bear to say something else, the bear will finish what it has to say and will wait for the user to trigger the bear to say something else. This is by my design; however, the uMP3 will support interrupting a MP3 file that is currently playing.

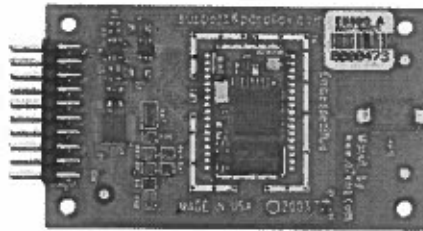
The code that the MP3 module uses is part of the IRRECEIVE.bs2 code found in the Appendix.

Bluetooth Communication

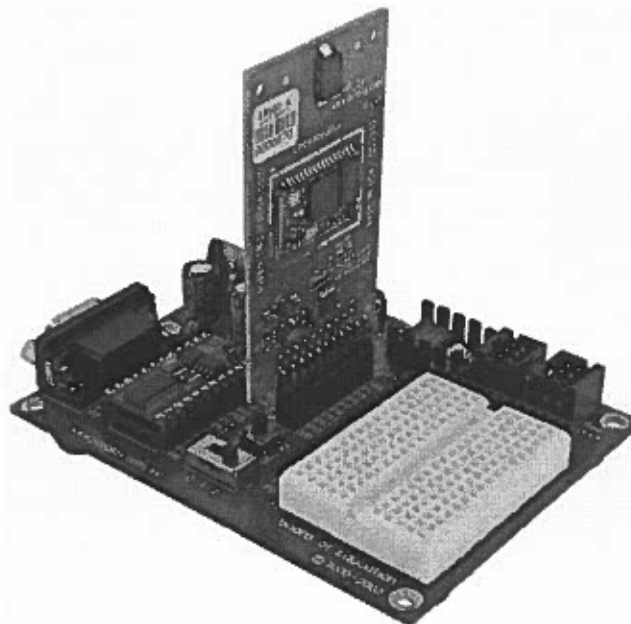
Bluetooth communication is used by the “My Museum” software program to trigger the bear to say different things in response to the “My Museum” program. My original time estimates for this project had only one month allocated to complete this module. This module took two months to complete due to unforeseen problems.

Bluetooth on the Bear

In order to establish Bluetooth communication between the bear and the PC, each device must have a Bluetooth module. The Bluetooth module on the bear is A7 Engineering's EmbeddedBlue eb500 Bluetooth Module shown below.



This module has two rows of ten pins each that plug into Board of Education's Application Module (AppMod) header as shown below.



Since having the board sticking out at this angle is not practical for the bear, I use long green wires between the eb500 and the AppMod to allow the eb500 to stay inside bear.

Bluetooth Module on PC

My computer does not have a Bluetooth module, so I used the Bluetooth module recommended by Parallax and the makers of eb500. They recommended the D-Link DBT-120 USB Bluetooth adapter.

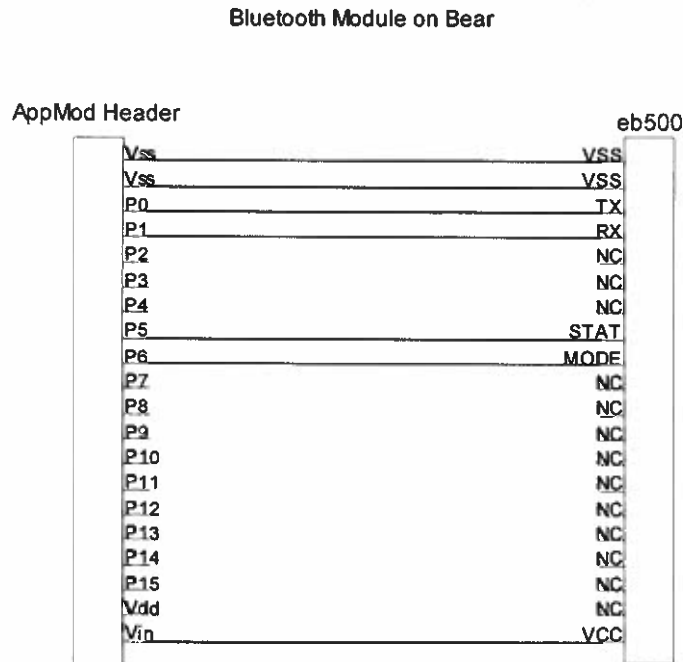


DBT-120

Bluetooth Blues

This Bluetooth module is much more complicated than it appears. To me, it made the infrared module seem like a “Hello, World!” program. After getting Bluetooth communication working on a spare Board of Education, I attempted to add the eb500 Bluetooth module to the bear’s main circuit board by plugging in wires from each of the eb500’s pins to each connection on the AppMod header. Once this was done, the bear’s main circuit board began to act erratically. I discovered that the AppMod header shared connections with the P0-P15 connections of the BASIC Stamp. The main circuit

board's erratic behavior went away when I only connected up the necessary wires between the eb500 and the AppMod header illustrated below.



Another problem I discovered was related to the two modes of the eb500. The eb500 has a command mode and a data mode. When the eb500 is first powered on, it is in command mode. The eb500 switches from command mode to data mode when the PC connects to it. Both the eb500 and the PC treat the Bluetooth connection as a serial connection, but when the eb500 goes into data mode, the eb500 blocks other serial connections from occurring on the bear's main circuit board. The uMP3 module also uses serial connections between itself and bear's main circuit board. But the uMP3 cannot communicate with the main circuit board when the eb500 is in data mode. My solution was to have the eb500 listen for a Bluetooth packet in data mode. The packet would contain the number of the MP3 file that the "My Museum" program needs the bear to say. As soon as the packet is received, the eb500 is switched from data mode to

command mode to allow the uMP3 to play the corresponding MP3 file. When the file has finished playing, the eb500 is switched back to data mode.

The BASIC Stamp code that the eb500 requires to establish a connection with a computer is not straight forward and requires the data entries in the EEPROM that I showed earlier in this chapter.

I believe that I may have had other problems in trying to get the eb500 to work and ended up ordering an eb600 carrier board for the eb500 so that I could hook the eb500 up to my computer and check it out like I did with the uMP3 modules.

Bluetooth Protocol

In order to get Bluetooth communication working in conjunction with IR communication on the main circuit board, I had to choose which communication would be the primary communication method of the bear. Since Bluetooth communication only occurs when the “My Museum” program is open, I changed the main circuit board to use the following protocol:

If eb500 has no Bluetooth connection, then listen for Infrared signals

If eb500 has a connection, then

- Get the Bluetooth packet and process it
- Switch the eb500 to command mode

- Play the uMP3 file for the Bluetooth packet
- Switch the eb500 to data mode
- Increment the packet index and send a reply packet

Repeat these steps

The general protocol used by the personal computer is as follows:

Make connection to the eb500 Bluetooth module

Transmit a packet to the eb500 repeatedly until a reply packet is received

Send incremented index from reply packet in next packet transmitted

Terminate connection to the eb500 Bluetooth

The code that the Bluetooth module uses is part of the IRRECEIVE.bs2 code found in the Appendix.

Microsoft Robotics Studio

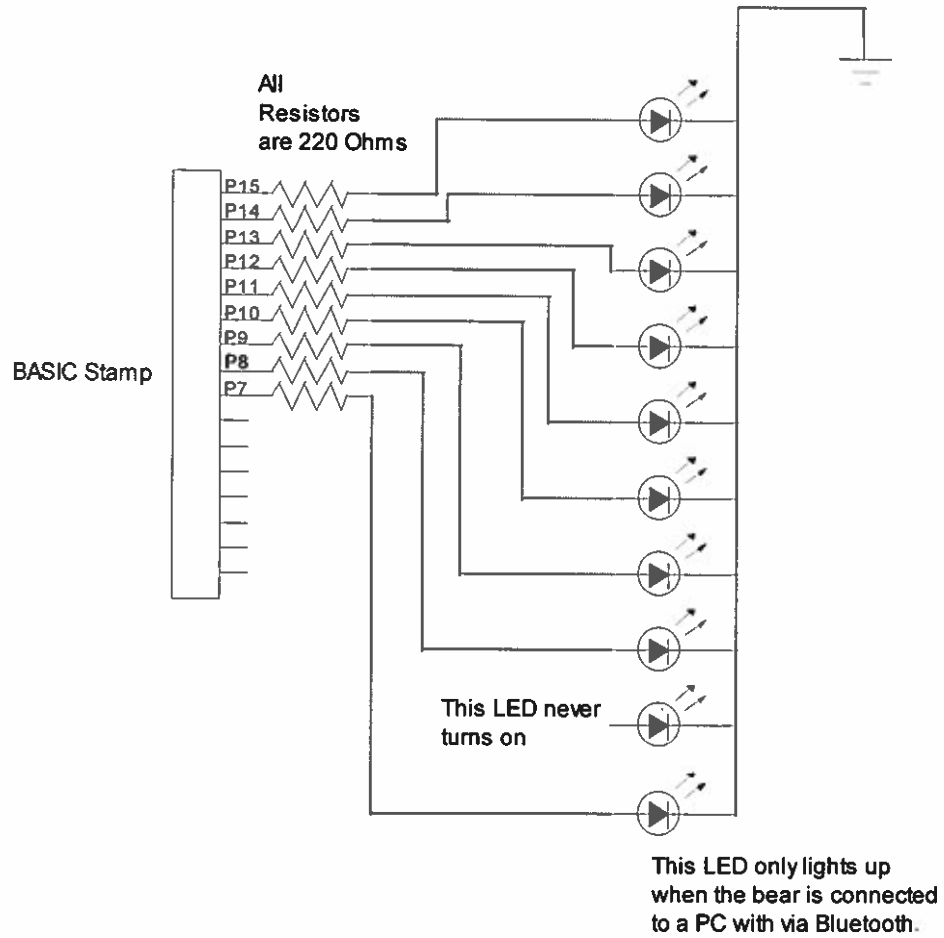
Parallax packages the eb500 module with their Bluetooth Boe-Bot Robot Kit for Microsoft Robotics Studio. Although I used the kit to get a better understanding of how to use Microsoft Visual C# to control a BASIC Stamp device via Bluetooth, I did not need the overhead that kit provided to control all the moving parts of a robot. I took the

core Bluetooth communication class out of the kit and modified it so that I could use it with the “My Museum” software.

System Status

I did not plan for this module to stay in the final Home and Away Bear design, but it worked so well that I left it in. This module consists of a row of LEDs on the main circuit board that displays status information to the user. When the bear is turned on, the main circuit board briefly lights up the LEDs to let you know it is on. Right before the IR receiver plays the MP3 file associated with an IR transmitter, it will light up the LEDs corresponding to the IR transmitter. The Bluetooth module lights up the last LED when it is connected to a computer and turns off the LED when it loses the Bluetooth connection. The lights corresponding to the file number that the Bluetooth module needs to play is also shown in the LEDs.

STATUS LEDS



Chapter 3.

Software

Software Requirements

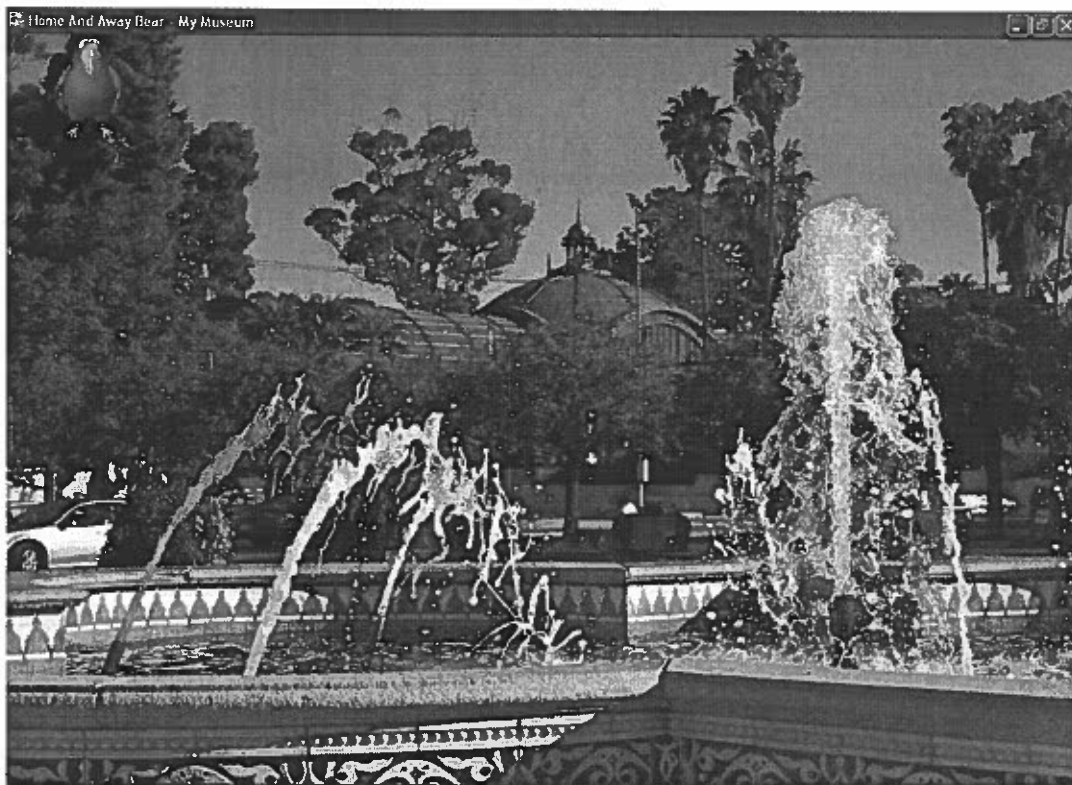
According to the project specification, this project is supposed to “wirelessly provide a unique computer experience where the device and the computer speak to each other in response to the user’s choices on the computer screen.” Not directly specified in the project specification is the use of C# and DirectX in developing the personal computer software. Microsoft Visual C# 2005 was used to write the software run by the user. However, GDI+ was used instead of DirectX because GDI+ is capable of generating 2D graphics, is easy to work with since it is native to Windows, and does not use graphics acceleration devices. GDI+ was a better fit to this project in order to keep the project focus on having the bear and the computer communicate with each other.

Software Design

The project specification mentioned that the bear and computer would appear to speak to each other. Trying to find the best scenario to have a character appear in the software without making this part of the project too complex was a struggle. I found the answer while flipping through Deitel’s C# for Programmers Second Edition book. “Microsoft Agent is a technology used to add interactive animated characters to Windows applications or Web pages. Microsoft Agent characters can speak and respond to user input via speech recognition and synthesis” (Deitel, 692-693). One of the Microsoft

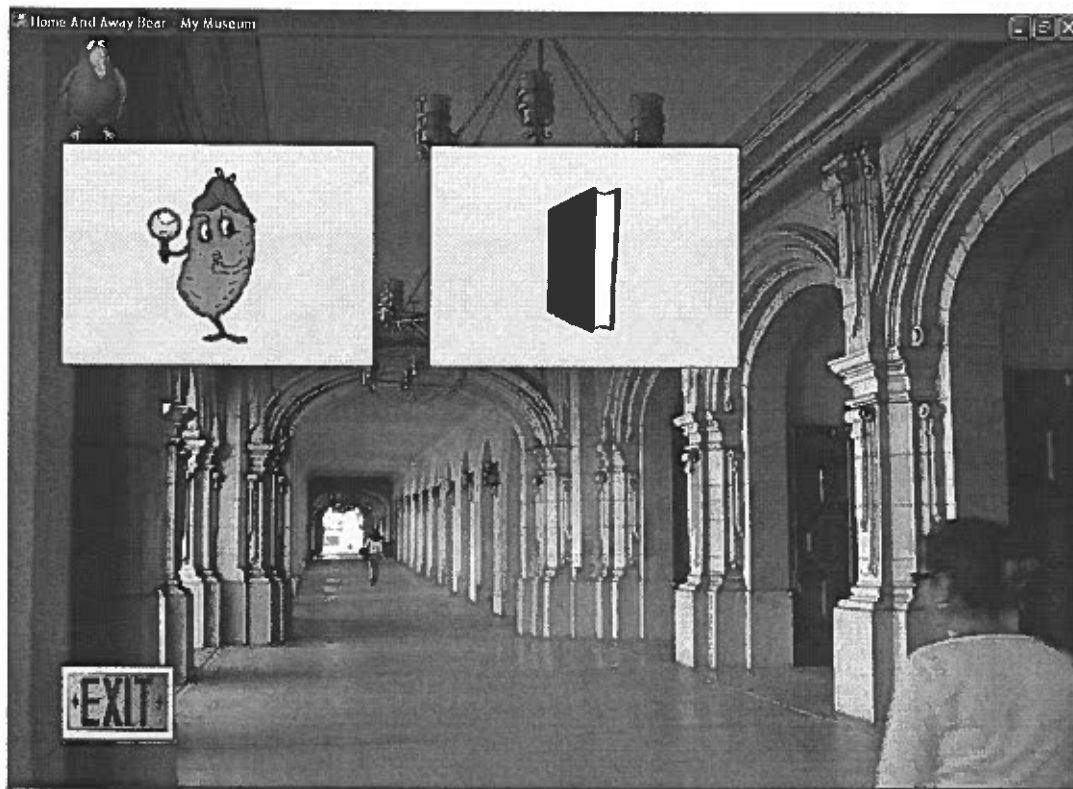
Agent characters is a parrot, which fit in perfectly in a scenario with a talking bear. However, I decided to let the bear do all of the talking and only use the animations of the parrot in order to establish a reason to use the bear with the software program.

Since the “Away” side of this project focuses on having the bear useful in a museum environment, I decided to use the same approach on this project’s “Home” side. The software program that the bear interacts with is called “My Museum”. When the bear is on and the user run “My Museum”, the bear will say “Welcome to My Museum. Click on Peedy the Parrot to go inside”. My voice is used for everything the bear says in “My Museum”. The parrot acts a tour guide and friend to take the user into the museum.

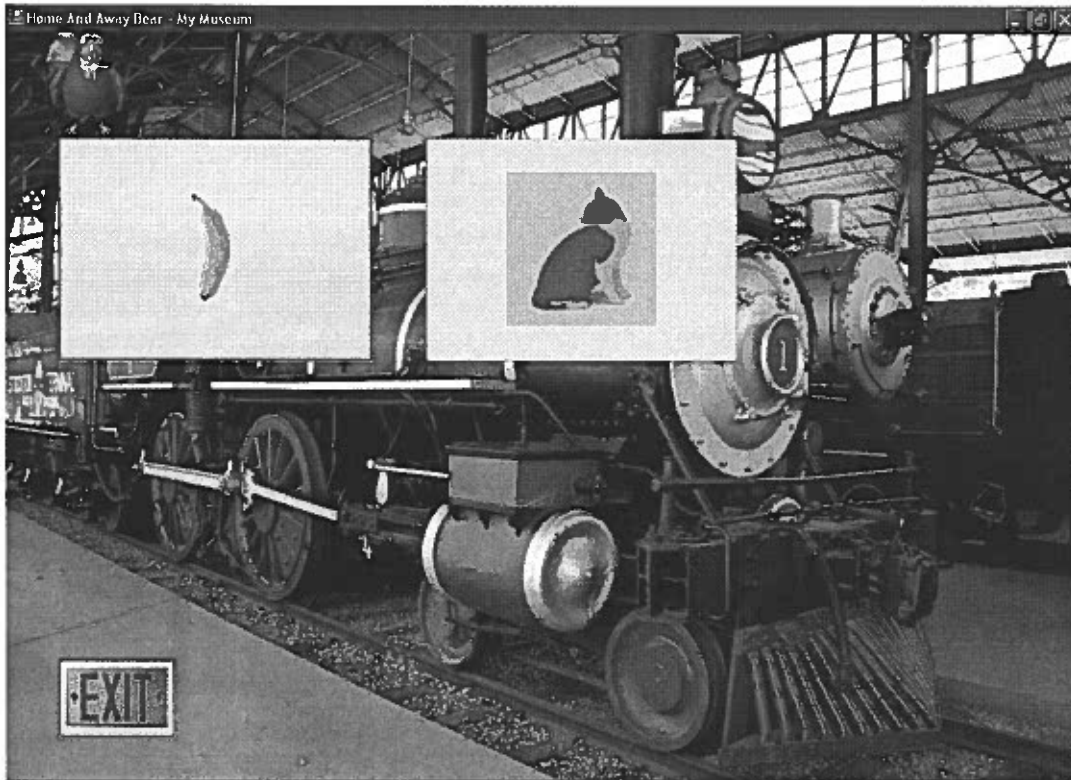


The outside of the museum and each section of the museum is accompanied by background relaying the part of the museum that we are in and thirty seconds of music that go with the background. All images and music in “My Museum” come from websites that freely give permission to use their media in commercial and non-commercial ventures. These websites are documented in the Appendix section of this document.

When the user is the museum lobby, the bear tells the user that they can do one of two things in the museum, they can either identify things that the bear says, or they can count books that they can read. The user is then prompted to make a selection. Once a selection is made, the parrot flies to the chosen section of “My Museum”.

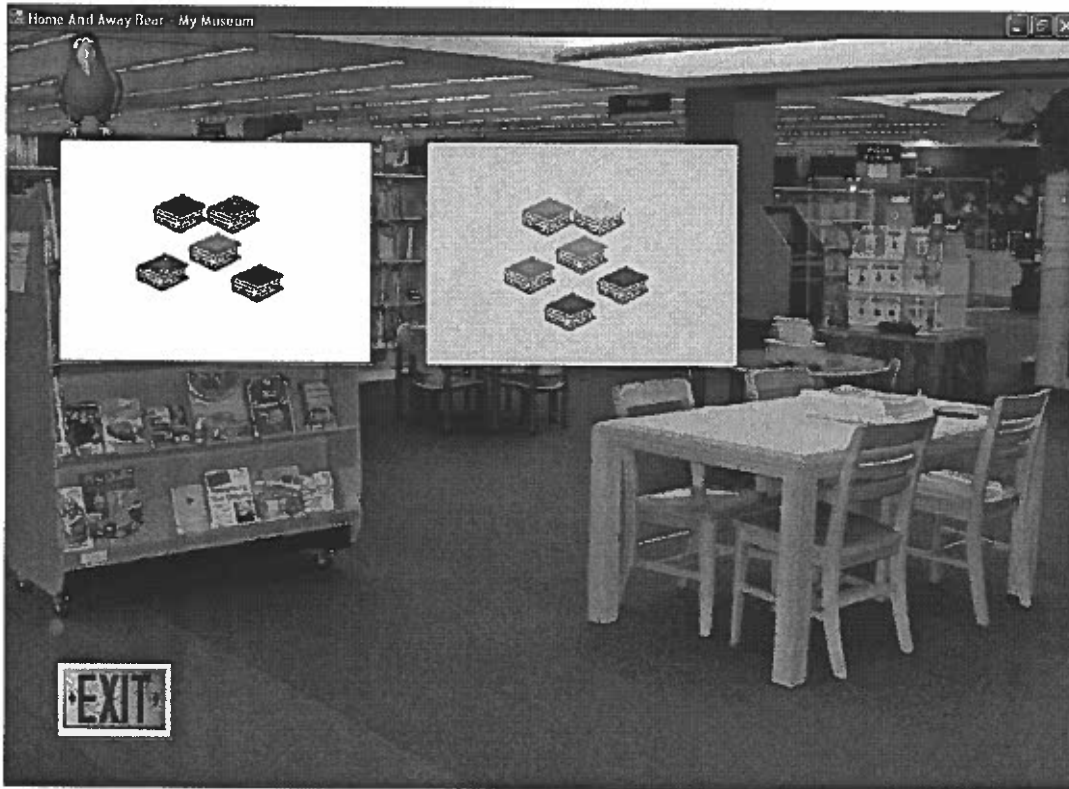


The screen below occurs during the “identify things” game. Two objects are shown on the screen and the bear says the name of one of the objects. The user has to choose the object that the bear identified. Once a selection is made, the incorrect object is grayed out and the bear tells the user if they are correct or incorrect. The parrot will smile if the user is correct and shake its head sadly if the user is not correct. When the bear has asked the user to identify three different things, the game is over and the parrot flies back to the museum lobby.

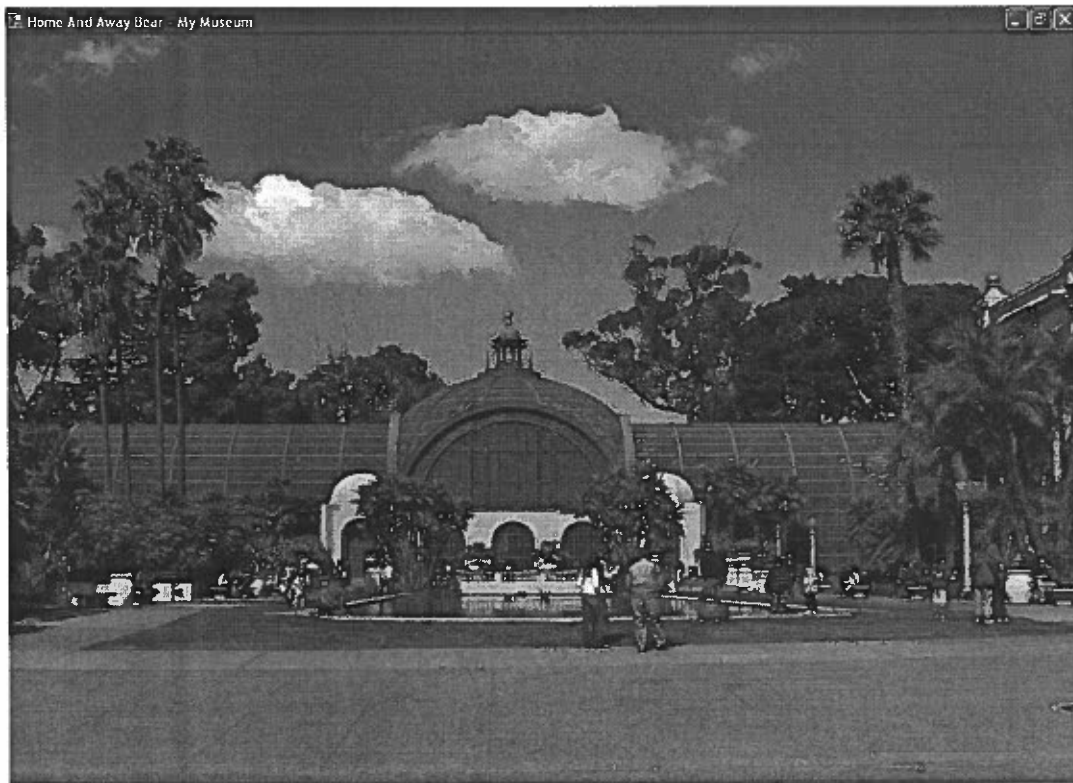


In the following screen, the user is playing the “count books we can read” game. In this game, the bear says the number of books that we can read. The user has to select the image with the correct number of books. In this picture, the user has selected the wrong

image and the parrot is shaking its head sadly. The user can choose the Exit button to go back to the museum lobby at any time.



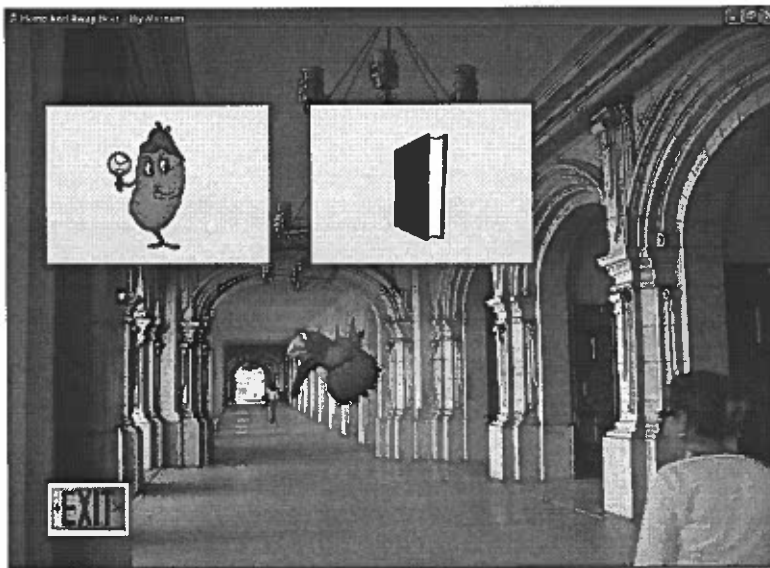
When the user is back in the museum lobby and chooses the Exit button, the parrot waves goodbye to the user and flies away. The bear then says, "Thanks for visiting My Museum. Now let's go explore your world". The user is then left outside the museum for the duration of the museum exit music.



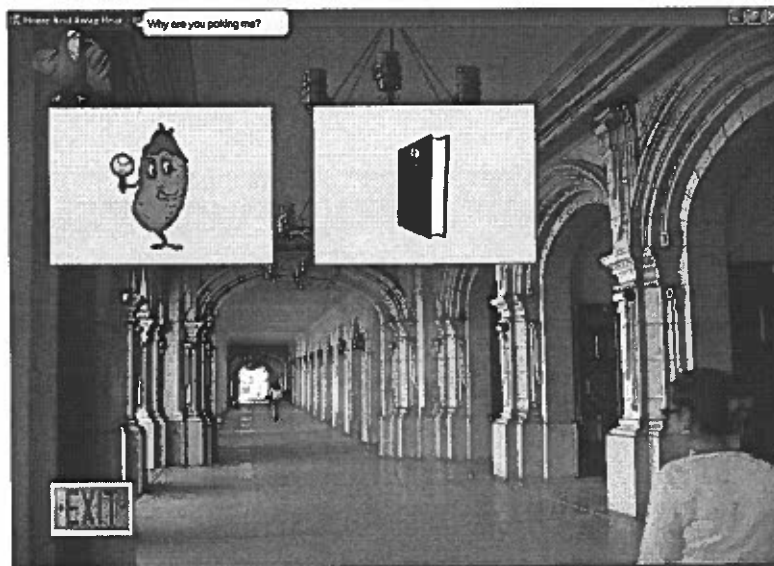
Other animations of the parrot used in this program occur as follows: when the parrot is clicked but is not outside the museum, the parrot acts surprised.



The parrot can fly around on any screen in the program by clicking an area on the screen not used by a button. The parrot will fly to that location on the screen.



The only time the parrot says anything is when he is double-clicked. The parrot acts confused and says, “Why are you poking me?” The parrot’s words are heard through the speaker and appear in a cartoon balloon next to the parrot.



Chapter 4.

Results

The Home and Away Bear meets the project specifications defined at the beginning of the project. It can process over 500 message variations. A museum can use over 250 infrared transmitters to have the bear say something in real-time about the museum's exhibit when the bear is within six feet of the exhibit and facing it. Verification is used by the infrared receiver to ensure that the bear talks about the correct exhibit with the infrared transmitter.

The size, weight, and form of the bear are similar to other devices that children carry around with them such as Interactive Pooh and Interactive Barney. The bear's built-in speaker focuses the bear's sound toward the user and not other people.

Only one BASIC Stamp Board of Education was required to support infrared communication, Bluetooth communication, and MP3 playback.

"My Museum" is a user-friendly software interface that uses pictures instead of printed words and provides a fun interactive experience with the bear. This project could easily be adapted for use in other languages by changing the MP3 files on the bear's SD card.

I have found Duracell batteries to be the best 9 volt batteries to use in this project. Inexpensive 9 volt batteries sometimes do not have enough power to run all of the components in the bear.

Chapter 5.

Conclusion

Summary

I have learned about wireless communication and how to configure microcontrollers to communicate wirelessly using infrared and Bluetooth technologies. I have a better handle on working with electronics and C# programming. It is possible to create a single device for the consumer that can work with both infrared and Bluetooth technology. Museums or other companies that choose to deploy Home and Away Bears will increase their exposure with the consumer by letting the consumer use the device to create a more personalized experience and allowing the consumer to bring home the experiences of that museum or other corporation and have the device work interactively with the user's home computer. This device would reinvent and improve the customer experience.

I have enjoyed having the opportunity to work on this project. I have wanted to do something useful with bear speakerphone for a long time. I appreciate having the opportunity to work on a project that does not reside total inside the computer and I encourage others in computer science that has an interest in parts of this project to study embedded systems.

Areas of Enhancement

The IR transmitter can be created without a BASIC Stamp microcontroller. This would reduce the cost of having to implement many IR transmitters in a museum.

The bear is still capable of eye and mouth movement. It should be possible to have a device feed off of the uMP3 module that used sounds to trigger eye and mouth movement.

Putting sensors in the bear's paws would allow more interaction with the bear's microcontroller when the bear is not near a computer or a museum.

The "My Museum" program should allow for the selection of a particular serial port.

A faster microcontroller with multithreading and handles interrupts would improve the response of the "My Museum" program. Parallax makes a Propeller chip that may do the job.

Add RF or GPS capabilities to the bear to give the bear more information about its environment.

Chapter 6.

Appendix

Microsoft Agent “Peedy” Actions

Acknowledge	LookLeft	MoveDown
Show	LookLeftBlink	MoveRight
GetAttention	LookLeftReturn	Idle3_3
Confused	LookUp	MoveLeft
GestureDown	LookUpBlink	Suggest
GestureUp	LookUpReturn	Wave
GestureLeft	Explain	Write
GestureRight	Sad	WriteReturn
Hearing_1	StopListening	Writing
Hearing_2	Surprised	Searching
Think	Uncertain	Idle2_1
Alert	Decline	Congratulate
RestPose	Idle1_1	Read
DontRecognize	Idle1_2	ReadReturn
Greet	Idle1_3	Reading
StartListening	Idle1_4	GetAttentionContinued
LookDown	Idle1_5	GetAttentionReturn
LookDownBlink	Hide	Blink
LookDownReturn	MoveUp	WriteContinued

Announce	LookDownLeftReturn	Process
ReadContinued	LookDownRight	Search
Pleased	LookDownRightBlink	LookRight
Idle2_2	LookDownRightReturn	LookRightBlink
Processing	LookUpLeftBlink	LookRightReturn
DoMagic1	LookUpLeft	Idle3_2
DoMagic2	LookUpLeftReturn	Hearing_3
Idle3_1	LookUpRight	Thinking
LookDownLeft	LookUpRightBlink	
LookDownLeftBlink	LookUpRightReturn	

Music in “My Museum”

Entering museum

http://www.pacdv.com/sounds/ambience_sounds.html (City Park)

All other music

<http://www.internetaudioguy.com/iag/freemusic/freemusic.htm>

Backgrounds in “My Museum”

Balboa Park, San Diego, CA

Museum entrance

<http://www.pdphoto.org/PictureDetail.php?mat=pdef&pg=8692>

Museum lobby

<http://www.pdphoto.org/PictureDetail.php?mat=pdef&pg=7253>

Museum exit

<http://www.pdphoto.org/PictureDetail.php?mat=pdef&pg=8486>

Columbus (Ohio) Metropolitan Library - Kid’s Section

“Count books” game

<http://www.morguefile.com/archive/?display=186838&>

Travel Town Train Museum in Los Angeles, CA

“Identify things” game

<http://www.morguefile.com/archive/?display=137206&>

Button GIFs in “My Museum”

Book image used in “count books” game

<http://www.horton-szar.net/clipart/books.php>

All other gifs on buttons

<http://www.animationlibrary.com/>

Sample MP3 of NASA Moon Landing

<http://www.freeinfosociety.com/site.php?postnum=460>

Code: IRSEND.bs2

```

' {$STAMP BS2}
' {$PBASIC 2.5}
'
' Student Name: Oscar A. Smalls, Jr.
' Program Name: IRSEND.bs2
' Date       : April 29, 2008
'
' -----[ Program Description ]-----
'
' This program displays the frequency of the 555 timer and sends data
' pulses corresponding to start bit, 0 bit, and 1 bit. Start pulse last
' for 2.4 ms. The 1 bit last for 1.2 ms. The 0 bit lasts for 0.6 ms.
'
' -----[ I/O Definitions ]-----

tPin      PIN    14          ' Transmit pin
fPin      PIN    15          ' Frequency sense pin

' -----[ Constants ]-----

DelayTime  CON    100
StartPulse CON   1200
DataPulse  CON    300
RestingState CON   23

' -----[ Variables ]-----

switches   VAR    INL          ' switch data
cycles     VAR    Word          ' Stores 555 timer frequency

' -----[ Initialization ]-----

Setup:

```

```

HIGH tPin                ' Enable 555 timer
COUNT fPin, 1000, cycles    ' Measure frequency
LOW tPin                  ' Disable 555 timer
DEBUG HOME, "Frequency = ", DEC5 cycles,    ' Display frequency
    " Hz", CR

DEBUG CR,
    "Switches  76543210", CR,
    "-----", CR,
    "Status    ....."

' -----[ Program Code ]-----

Main:
DO
    DEBUG CRSRXY, 10, 4, BIN8 switches
    GOSUB Send_Switches
    PAUSE DelayTime
LOOP

' -----[ Subroutines ]-----

Send_Switches:

LOW tPin                ' 555 signal off
PAUSE RestingState      ' Rest between messages
PULSOUT tPin, StartPulse    ' Start pulse
PULSOUT tPin, switches.BIT0 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT1 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT2 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT3 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT4 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT5 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT6 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT7 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT0 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT1 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT2 * DataPulse + DataPulse

```

```
PULSOUT tPin, switches.BIT3 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT4 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT5 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT6 * DataPulse + DataPulse
PULSOUT tPin, switches.BIT7 * DataPulse + DataPulse

RETURN
```

Code: IRRECEIVE.bs2

```

' {$STAMP BS2}
' {$PBASIC 2.5}
'
' Student Name: Oscar A. Smalls, Jr.
' Program Name: IRRECEIVE.bs2
' Date       : April 29, 2008

' -----[ Program Description ]-----
'
' This program is runs on the main circuit board of the Home and Away Bear.
' It contains an IR detector sends data to the BASIC Stamp. The BASIC
' Stamp analyzes the data to determine if it came from an IR transmitter.
' If the data is valid, it plays the MP3 file associated with the IR
' transmitter.
'
' This program also uses the eb500 to listen for a bluetooth conection. If
' there is a bluetooth connection, it stops waiting for data from the
' IR detector.
'
' -----
' PNA4612 IR detector
' =====
' 1 - Blue      220 Ohms - P8
' 2 - White     Vss
' 3 - Red       Vdd

' -----
' Initialization of UMP3
' =====
' UMP3
' =====
' PWR
' +      Vdd
' G      Vss
'

```

```

' TTL-232      USB2SER
'   G          VSS
'   T          RX
'   R          TX
'
' HyperTerminal initialization procedure
' -----
' -----
' UMP3          Board of Education
' =====
' PWR
'   +          Vdd
'   G          Vss
'
' TTL-232
'   T          P3
'   R          P4
'
' -----[ I/O Definitions ]-----
eb500_Output   PIN    0
eb500_Input    PIN    1
IR_Receive     PIN    2      ' I/O Pin to IR detector Output
uMP3_Transmit  PIN    3      ' uMP3: set the pin where the uMP3 "T" pin is connected
uMP3_Receive   PIN    4      ' uMP3: set the pin where the uMP3 "R" pin is connected
eb500_Connect  PIN    5      ' 0=not connected; 1=connected
eb500_Mode     PIN    6      ' 0=Command Mode; 1=Data Mode
eb500_LED      PIN    7      ' Lit when eb500 is connected
IR_LEDs        VAR    OUTH   ' LED outputs (Pins 8 through 15)
IR_LEDsDirs    VAR    DIRH   ' DIRS control For IR_LEDs

' -----[ Constants ]-----

' IR Constants
ThresholdStart CON    1000

```



```

ThresholdEdge  CON    300
UnderStart     CON    675
OverStart      CON    1125

```

```
' uMP3 Constants
```

```

N9600T        CON    84      ' uMP3: Set the 9600 8 bit no parity TRUE
serStrSize     CON     9

```

```
' -----[ Variables ]-----
```

```
' IR Variables
```

```

irPulse        VAR    Word           ' Stores pulse widths
switches       VAR    Byte           ' Stores remote code
switchesInput  VAR    Byte
switchesCheck  VAR    Byte
switchesHold   VAR    Byte

```

```
' uMP3 Variables
```

```

serStr         VAR    Byte(serStrSize)
serdata        VAR    Byte      ' Set aside for input from the uMP3

```

```
' eb500 Variables
```

```

' Buffer array not declared as buffer VAR Word(5) for SERIN functionality.
' It can still be accessed as buffer(0), buffer(1), etc. However,
' buffer0, buffer1, etc. should be used in SERIN commands with variations
' of WAIT.

```

```

buffer0        VAR    Byte           ' Buffer - Start char = $ff
buffer1        VAR    Byte           ' Message Index value
buffer2        VAR    Byte           ' Command
buffer3        VAR    Byte           ' Argument 1 (return data 1)
buffer4        VAR    Byte           ' Argument 2 (return data 2)
buffer         VAR    buffer0        ' For standard array indexing

pointer        VAR    Byte           ' EEPROM pointer
msgIndex       VAR    Byte           ' EEPROM message index

```

```

routine      VAR      Nib      ' Routine selector
counter      VAR      Nib
index        VAR      Nib

' -----[ EEPROM DATA ]-----

' eb500 data
RequestConnect      DATA $FF, 0, 1, 0, 0
ConnectionGranted   DATA $FF, 0, 2, 0, 0
RequestCommand      DATA $FF, 0, 3, 0, 0

' -----[ Initialization ]-----

Setup:

IR_LEDSdirs = %11111111      ' make IR_LEDs outputs
IR_LEDS = %11111111      'Light up the IR_LEDs for the IR switches

SEROUT uMP3_Receive,N9600T,[CR]      'Send an INIT CR to the uMP3
GOSUB uReadMP3      'Check for response from uMP3
GOSUB uVolReset      'Reset the uMP3 volume to FULL
GOSUB uStop      'Tell the unit to STOP in case it's playing
GOSUB uLoopCount
GOSUB Play_Title_Sequence

'  DEBUG CLS,
'      "Switches 76543210", CR,
'      "-----", CR,
'      "Status      ...."

GOSUB Initialize_IR_Variables

' -----[ Program Code ]-----

Main:

DO

    IF eb500_Connect = 1 THEN ' eb500 has bluetooth connection

```

```

    IF eb500_LED = 0 THEN
        GOSUB Initialize_IR_Variables
        HIGH eb500_LED
        GOSUB Reset_eb500
    ENDIF

    GOSUB Process_eb500_communication
ELSE ' eb500 does not have bluetooth connection
    LOW eb500_LED
    GOSUB Process_IR_communication
ENDIF

LOOP
END

```

' -----[IR Subroutines]-----

Initialize_IR_Variables:

```

    serStr(0) = "0"
    switches = 0
    switchesHold = 0
    IR_LEDs = 0
    RETURN

```

Process_IR_communication:

```

    DEBUG "Process_IR_communication",CR
    GOSUB Listen_for_Switches
    IF switches <> switchesHold THEN
        switchesHold = switches
        GOSUB Display_Switches_IR_LEDs
        GOSUB Process_Switches
    ENDIF
    RETURN

```

Listen_for_Switches:

```

    switchesInput = 0
    switchesCheck = 0

```

```
' Check to see if a falling edge has occurred

RCTYPE IR_Receive, 1, irPulse

IF irPulse > ThresholdStart THEN

    ' Measure start pulse.

    RCTYPE IR_Receive, 0, irPulse

    IF irPulse >= UnderStart AND irPulse <= OverStart THEN

        ' Get data bit pulses.

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT0 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT1 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT2 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT3 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT4 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT5 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT6 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesInput.BIT7 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesCheck.BIT0 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesCheck.BIT1 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesCheck.BIT2 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesCheck.BIT3 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesCheck.BIT4 = 1

        RCTYPE IR_Receive, 0, irPulse

        IF irPulse > ThresholdEdge THEN switchesCheck.BIT5 = 1

        RCTYPE IR_Receive, 0, irPulse
```

```

    IF irPulse > ThresholdEdge THEN switchesCheck.BIT6 = 1

    RCTIME IR_Receive, 0, irPulse

    IF irPulse > ThresholdEdge THEN switchesCheck.BIT7 = 1

    IF switchesInput = switchesCheck THEN
        switches = switchesInput
    ENDIF

ENDIF

ENDIF

RETURN

'Display_Switches_Debug:
'  DEBUG CRSRXY, 10, 2, BIN8 switches
'  RETURN

Process_Switches:

    IF switches.BIT7 = 1 THEN serStr(1) = "1" ELSE serStr(1) = "0"
    IF switches.BIT6 = 1 THEN serStr(2) = "1" ELSE serStr(2) = "0"
    IF switches.BIT5 = 1 THEN serStr(3) = "1" ELSE serStr(3) = "0"
    IF switches.BIT4 = 1 THEN serStr(4) = "1" ELSE serStr(4) = "0"
    IF switches.BIT3 = 1 THEN serStr(5) = "1" ELSE serStr(5) = "0"
    IF switches.BIT2 = 1 THEN serStr(6) = "1" ELSE serStr(6) = "0"
    IF switches.BIT1 = 1 THEN serStr(7) = "1" ELSE serStr(7) = "0"
    IF switches.BIT0 = 1 THEN serStr(8) = "1" ELSE serStr(8) = "0"

    SEROUT uMP3_Receive,N9600T,["PC F /",STR serStr\serStrSize,".mp3",CR]
'  DEBUG CRSRXY, 0, 4, "PC F /",STR serStr\serStrSize,".mp3",CR
    DEBUG "PC F /",STR serStr\serStrSize,".mp3",CR
    GOSUB uWaitDone

RETURN

Display_Switches_IR_LEDs:

    IF switches.BIT7 = 1 THEN IR_LEDs.BIT0 = 1 ELSE IR_LEDs.BIT0 = 0
    IF switches.BIT6 = 1 THEN IR_LEDs.BIT1 = 1 ELSE IR_LEDs.BIT1 = 0
    IF switches.BIT5 = 1 THEN IR_LEDs.BIT2 = 1 ELSE IR_LEDs.BIT2 = 0
    IF switches.BIT4 = 1 THEN IR_LEDs.BIT3 = 1 ELSE IR_LEDs.BIT3 = 0

```

```

IF switches.BIT3 = 1 THEN IR_LEDs.BIT4 = 1 ELSE IR_LEDs.BIT4 = 0
IF switches.BIT2 = 1 THEN IR_LEDs.BIT5 = 1 ELSE IR_LEDs.BIT5 = 0
IF switches.BIT1 = 1 THEN IR_LEDs.BIT6 = 1 ELSE IR_LEDs.BIT6 = 0
IF switches.BIT0 = 1 THEN IR_LEDs.BIT7 = 1 ELSE IR_LEDs.BIT7 = 0

RETURN

```

```

' -----[ uMP3 Subroutines ]-----

```

```

uStop:                                'Stop the player

```

```

'  DEBUG "Send STOP command to player:",CR

```

```

'  DEBUG "PC S",CR

```

```

SEROUT uMP3_Receive,N9600T,["PC S",CR]

```

```

GOSUB uReadMP3

```

```

PAUSE 500

```

```

RETURN

```

```

uVolReset:                            'Reset the volume to "full"

```

```

'  DEBUG "Send VOLUME command to player:",CR

```

```

'  DEBUG "ST V",DEC 0,"      ",CR

```

```

SEROUT uMP3_Receive,N9600T,["ST V ",DEC 0,CR]

```

```

GOSUB uReadMP3

```

```

PAUSE 500

```

```

RETURN

```

```

uLoopCount:

```

```

'  DEBUG "Set Loop Count:",CR

```

```

'  DEBUG "ST O 1",CR

```

```

SEROUT uMP3_Receive,N9600T,["ST O 1",CR]

```

```

GOSUB uReadMP3

```

```

PAUSE 500

```

```

RETURN

```

```

uWaitDone:                            ' Wait here till the current song finishes

```

```

DEBUG "Waiting for mp3 to end...",CR

```

```

WAITING:

```

```

SEROUT uMP3_Receive,N9600T,["PC Z",CR]      ' Check the status of the uMP3

```

```

SERIN uMP3_Transmit, N9600T, 1000, noData1, [serdata] ' Get the response

```

```

' DEBUG serdata,CR                                ' Debug to see what the uMP3 is returning
IF serdata = "S" THEN RETURN                        ' If the player has finished playing the file,
jump to the start
noData1:
PAUSE 100                                           ' otherwise, kill some time
GOTO WAITING

uReadMP3:                                           'Read command response from uMP3 unit
SERIN uMP3_Transmit, N9600T, 1000, No_Respl, [serdata]
RETURN
No_Respl:                                           'Tell user there is no reply from uMP3
DEBUG "No response from uMP3!",CR
RETURN

Play_Title_Sequence:
SEROUT uMP3_Receive,N9600T,["PC F /0000000000.mp3",CR]
GOSUB uWaitDone
RETURN

' -----[ eb500 Subroutines ]-----

Process_eb500_communication:
'  DEBUG "Process_eb500_communication",CR

IF eb500_Connect = 0 THEN RETURN ' eb500 has lost bluetooth connection

SERIN eb500_Output, 84, 5, Process_eb500_communication, ' Get next command
[WAITSTR buffer \ 2, buffer2, buffer3, buffer4]

IF buffer2 = 192 THEN                               ' Handle restart req from PC
  msgIndex = 0
  GOTO Request_Connect
ENDIF

LOOKDOWN buffer2,[96], routine

ON routine GOSUB Play_File

```

```

' Increment message index for reply. Next message from PC has to use
' reply's buf[1].
msgIndex = msgIndex + 1
buffer1 = msgIndex
DEBUG "buffer1=",DEC buffer1,CR
SEROUT eb500_Input, 84, [STR buffer \5]

RETURN

```

Reset_eb500:

```

DEBUG "Reset_eb500",CR

msgIndex = 0

GOSUB Ram_Clear

DEBUG "Request_Connect",CR
Request_Connect:
    pointer = RequestConnect
    GOSUB Get_Packet
    SEROUT eb500_Input, 84, [STR buffer \5]

DEBUG "Confirm_Connect",CR
Confirm_Connect:
    pointer = ConnectionGranted
    GOSUB Get_Packet
    Wait_For_Confirm:
        SEROUT eb500_Input, 84, [255, 0, 1, 0, 0]
        SERIN eb500_Output, 84, 20, Wait_For_Confirm, [WAITSTR buffer \ 2,
            buffer2, buffer3, buffer4]

DEBUG "Request_Packet",CR
Request_Packet:
    msgIndex = msgIndex + 1
    pointer = RequestCommand
    GOSUB Get_Packet
    SEROUT eb500_Input, 84, [STR buffer \5]

RETURN

```


Play_File:

DEBUG "Play_File",CR

' DEBUG CRSRXY, 10, 3, BIN8 msgIndex

'Switch to Command Mode

LOW eb500_Mode

SERIN eb500_Output,84,[WAIT(CR,">")]

DEBUG "In Command Mode",CR

IF buffer3 = 0 THEN serStr(0)= "0" ELSE serStr(0) = "1"

switches = buffer4

GOSUB Display_Switches_IR_LEDs

GOSUB Process_Switches

GOSUB Initialize_IR_Variables

'Return to Data Mode

HIGH eb500_Mode

PAUSE 300

DEBUG "In Data Mode",CR

' GOSUB Reset_eb500

RETURN

' Go to resume routine

Get_Packet:

DEBUG "Get_Packet",CR

FOR index = 0 TO 4

READ pointer + index, buffer(index)

DEBUG DEC buffer(index),CR

NEXT

buffer(1) = msgIndex

RETURN

Ram_Clear:

W0 = 0

```
W1 = 0
W2 = 0
W3 = 0
W4 = 0
W5 = 0
W6 = 0
W7 = 0
W8 = 0
W9 = 0
W10 = 0
W11 = 0
W12 = 0
RETURN
```

Code: Bluetooth.cs

```
// Student Name: Oscar A. Smalls, Jr.
// Program Name: Bluetooth.cs
// Date       : April 26, 2008
// This program is used to communicate with the eb500 module on
// the Home and Away Bear.

using System;
using System.IO.Ports;
using System.Collections.Generic;
using System.Text;

namespace MyMuseum
{
    public class Bluetooth
    {
        SerialPort _serialPort;

        bool connected = false;
        bool running = false;

        public bool Running
        {
            get { return running; }
            set { running = value; }
        }

        byte msgCnt;

        bool handshake = false; // Handshake after BT connection

        public Bluetooth()
        {
        }

        ~Bluetooth()
        {
        }
    }
}
```

```
        Close();
    }

    public bool Connect(int serialPort, out string errorMessage)
    {
        if (serialPort <= 0)
        {
            errorMessage = "The serial port is not configured!";
            return false;
        }
        if (connected)
        {
            Close();
        }
        try
        {
            msgCnt = 0;

            string ComPort = "COM" + serialPort.ToString();
            _serialPort = new SerialPort(ComPort, 9600, Parity.None, 8,
StopBits.One);

            _serialPort.Open();
            _serialPort.DiscardInBuffer();
            errorMessage = string.Empty;
            connected = true;
            return true;
        }
        catch (Exception ex)
        {
            errorMessage = string.Format("Error connecting to COM{0}: {1}",
serialPort, ex.Message);
            return false;
        }
    }

    public void Close()
    {
        if (!connected)
```

```
        return;

        connected = false;

        if (running)
        {
            running = false;
        }

        if (_serialPort != null && _serialPort.IsOpen)
        {
            _serialPort.Close();
            _serialPort = null;
        }
    }

    public void Start()
    {
        running = true;
    }

    public void Stop()
    {
        running = false;
    }

    public void Play(int fileType, int fileName)
    {
        if (running)
        {
            if (!handshake)
            {
                handshake = WaitForConnect();
            }

            if (handshake)
            {
                PlayFile(fileType, fileName);
            }
        }
    }
}
```

```
}

private void PlayFile(int fileType, int fileName)
{
    byte[] buf = new byte[5];
    buf[2] = 96;
    buf[3] = (byte) fileType;
    buf[4] = (byte) fileName;
    SendPackets(ref buf);
}

private bool WaitForConnect()
{
    byte[] buf = new byte[5];
    buf[1] = 0;
    buf[2] = 192;
    SendPackets(ref buf);
    do
    {
        if (!running)
        {
            break;
        }

        buf[2] = (byte)2;
        SendPackets(ref buf);
    } while (buf[2] != (byte)3);
    msgCnt = (byte)buf[1];
    return true;
}

private bool SendPackets(ref byte[] packet)
{
    if (packet.Length != 5) throw new SystemException("Invalid packet length!");
    packet[0] = 255; // Start byte
    packet[1] = msgCnt; // Message index, incremented by Basic Stamp
                        // must be used in next message that is sent.
    while (_serialPort.BytesToRead < 5)
```

```
{
    _serialPort.Write(packet, 0, 5);
}

_serialPort.Read(packet, 0, 5); // get Basic Stamp's reply
msgCnt = packet[1]; // Get next message count
if (packet[2] == 1) handshake = false; // Basic Stamp requests reconnect
return true;
}
}
}
```

Code: CountGame.cs

```
// Student Name: Oscar A. Smalls, Jr.
// Program Name: CountGame.cs
// Date       : April 26, 2008

// This class is a decendent of the ImageGame class and is used
// to populate the ImageGame with picture resources and the MP3 file number
// of those resources on the Home and Away Bear.

using System;
using System.Collections.Generic;
using System.Text;

namespace MyMuseum
{
    public class CountGame : ImageGame
    {
        private Image[] imageList =
        {
            new Image("count1", 201),
            new Image("count2", 202),
            new Image("count3", 203),
            new Image("count4", 204),
            new Image("count5", 205),
            new Image("count6", 206),
            new Image("count7", 207),
            new Image("count8", 208),
            new Image("count9", 209)
        };

        public CountGame()
        {
            Register(imageList);
        }
    }
}
```


Code: IdentifyGame.cs

```
// Student Name: Oscar A. Smalls, Jr.
// Program Name: IdentifyGame.cs
// Date       : April 26, 2008

// This class is a decendent of the ImageGame class and is used
// to populate the ImageGame with picture resources and the MP3 file number
// of those resources on the Home and Away Bear.

using System;
using System.Collections.Generic;
using System.Text;

namespace MyMuseum
{
    public class IdentifyGame : ImageGame
    {
        private Image[] imageList =
        {
            new Image("apple", 101),
            new Image("banana", 102),
            new Image("butterfly", 103),
            new Image("camera", 104),
            new Image("cat", 105),
            new Image("dog", 106),
            new Image("bull", 107),
            new Image("cow", 108),
            new Image("sun", 109),
            new Image("moon", 110)
        };

        public IdentifyGame()
        {
            Register(imageList);
        }
    }
}
```


Code: Image.cs

```
// Student Name: Oscar A. Smalls, Jr.
// Program Name: Image.cs
// Date       : April 26, 2008

// This class holds the picture resource name and the MP3 file number
// of corresponding to the resource

using System;
using System.Collections.Generic;
using System.Text;

namespace MyMuseum
{
    public class Image
    {
        private string name;
        private int number;

        public Image(string imageName, int imageNumber)
        {
            name = imageName;
            number = imageNumber;
        }

        public string Name
        {
            set { name = value; }
            get { return name; }
        }

        public int Number
        {
            set { number = value; }
            get { return number; }
        }
    }
}
```

f

Code: ImageGame.cs

```
// Student Name: Oscar A. Smalls, Jr.
// Program Name: ImageGame.cs
// Date       : April 26, 2008

// This ancestor class is responsible for handling the games that are
// played in the museum. A game consists of three rounds. Each round
// has two images that are displayed to the user. The user will need
// to pick the correct image based on feedback from the bear.

using System;
using System.Collections.Generic;
using System.Text;

namespace MyMuseum
{
    public class ImageGame
    {
        private Image[] imageList; // provided by CountGame or IdentifyGame

        private int[] answerList = new int[3]; // the answers for the three rounds

        public void Register(Image[] list) // used by CountGame and IdentifyGame
        {
            imageList = list;
        }

        public void NewGame()
        {
            Random randomNumbers = new Random();

            // shuffle the images in the list
            for (int first = 0; first < imageList.Length; first++)
            {
                int second = randomNumbers.Next(imageList.Length);
                Image temp = imageList[first];
                imageList[first] = imageList[second];
            }
        }
    }
}
```

```
        imageUrl[second] = temp;
    }

    // select the image that will be the answer for each round
    int counter = 0;
    for (int i = 0; i < 6; i = i + 2)
    {
        answerList[counter++] = i + randomNumbers.Next(2);
    }

}

// return the name of the image on the left
// this is used to compare to the correct answer
// argument to this method is the round of the game
public string LeftName(int i)
{
    string myString;

    switch (i)
    {
        case 0:
            myString = imageUrl[0].Name;
            break;
        case 1:
            myString = imageUrl[2].Name;
            break;
        case 2:
            myString = imageUrl[4].Name;
            break;
        default:
            myString = "";
            break;
    }

    return myString;
}
```

```
// return the name of the image on the right
// this is used to compare to the correct answer
// argument to this method is the round of the game
public string RightName(int i)
{
    string myString;

    switch (i)
    {
        case 0:
            myString = imageList[1].Name;
            break;
        case 1:
            myString = imageList[3].Name;
            break;
        case 2:
            myString = imageList[5].Name;
            break;
        default:
            myString = "";
            break;
    }

    return myString;
}

// return the name of the correct image
// used to compare to the user's answer
public string AnswerName(int i)
{
    return imageList[answerList[i]].Name;
}

// return the MP3 file number of the correct image
// used to have the bear say the name of this image
public int AnswerNumber(int i)
{

```



```
        return imageList[answerList[i]].Number;
    }
}
}
```

Code: MyMuseum.cs

```
// Student Name: Oscar A. Smalls, Jr.
// Program Name: MyMuseum.cs
// Date       : April 26, 2008

// This is the main program. It uses a delegate to create a separate thread for
// the bluetooth object. The bluetooth object is used to make the bear say something.
// When a script uses the Say method, it is at the end of that script.
// After the bear talks, the returncode variable is set with the file number
// that the bear called. The timer in this program looks at the returncode and
// performs an action based on the returncode.

// File numbers of what the bear can say:
//
// 1: WELCOME TO MY MUSUEM. CLICK ON PEEDY THE PARROT TO TAKE US INSIDE
// 2: IN MY MUSEUM, WE CAN
// 3: IDENTIFY DIFFERENT THINGS
// 4: OR
// 5: COUNT BOOKS WE CAN READ
// 6: YOU CHOOSE
// 7: THIS IS A GREAT GAME.
//    I WILL SHOW YOU TWO THINGS
//    AND SAY THE NAME OF ONE OF THEM.
//    PICK THE THING THAT I SAY
// 8: READING IS FUN. I WILL SAY THE NUMBER
//    OF BOOKS THAT WE HAVE TO READ.
//    CHOOSE THE SIDE WITH THE CORRECT
//    NUMBER OF BOOKS.
// 9: THIS IS THE SIDE WITH _____
// 10: THANKS FOR VISTING MY MUSUEM.
//    NOW LET'S GO EXPLORE YOUR WORLD
// 11: CHOOSE THE SIDE WITH _____
// 12: THAT'S CORRECT
// 13: THAT'S NOT RIGHT
// 14: PEEDY LOVES TO READ
// 15: PEEDY LOVES TO IDENTIFY DIFFERENT THINGS
// 16: PEEDY LOVES TO COUNT BOOKS
```

// 17: PEEDY LOVES TO WRITE ABOUT DIFFERENT THINGS

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using System.IO;
using System.Resources;
using System.Reflection;
using System.Drawing.Drawing2D;
using System.Media;
using System.Threading;
using System.Runtime.Remoting.Messaging;

namespace MyMuseum
{
    // these are used to determine which area of the museum that the user is in
    enum Background
    {
        Museum_entrance, Museum_lobby, Museum_count, Museum_identify, Museum_exit
    }

    // this spins off a separate thread of the bluetooth object
    public delegate void MyDelegate(ref Bluetooth_control, int fileNumber);

    public partial class MyMuseum : Form
    {
        static private int returncode = 0; // the filename last played by the mp3
        private Button btnLeftSelection; // the button for the left image
        private Button btnRightSelection; // the button for the right image
        private Button btnExitDoor; // the button for the exit sign image
        private int threadSleep = 300; // used to suspend this thread to give
        // the bluetooth object thread time to
        complete
    }
}
```

```

// This is called when the bluetooth object thread has completed
static private void TalkComplete(IAsyncResult itfAR)
{
    // Retrieve the informational object
    returncode = (int)itfAR.AsyncState;
}

// MyDelegate calls this in a separate thread
static private void Talk(ref Bluetooth _control, int fileNumber)
{
    _control.Play(1, fileNumber);
    return;
}

Random randomNumbers = new Random();

private Bluetooth _control = null; // this is the bluetooth object
int SerialPort = 40; // bluetooth is connected to this port on this computer
string errorMessage;
Selection leftSelection; // left button object
Selection rightSelection; // right button object
Selection exitDoor; // exit button object
ResourceManager rm;
GraphicsPath myPath = new GraphicsPath();
Background imageBackground;
SoundPlayer soundPlayer = new SoundPlayer();
IdentifyGame identifyGame = new IdentifyGame();
CountGame countGame = new CountGame();
int round = 0;
MyDelegate b = new MyDelegate(Talk);

// current agent object
private AgentObjects.IAgentCtlCharacter agent = null;

public MyMuseum()
{
    InitializeComponent();
}

```

```

    btnLeftSelection = new Button();
    btnLeftSelection.Click += new EventHandler(btnLeftSelection_Click);
    Controls.Add(btnLeftSelection);

    btnRightSelection = new Button();
    btnRightSelection.Click += new EventHandler(btnRightSelection_Click);
    Controls.Add(btnRightSelection);

    btnExitDoor = new Button();
    btnExitDoor.Click += new EventHandler(btnExitDoor_Click);
    Controls.Add(btnExitDoor);

    _control = new Bluetooth();
    _control.Connect(SerialPort, out errorMessage);
    _control.Start();

    rm = new ResourceManager("MyMuseum.MyCustomResources",
        Assembly.GetExecutingAssembly());

    leftSelection = new Selection(btnLeftSelection, rm, 50, 100, 300, 214);
    rightSelection = new Selection(btnRightSelection, rm, 400, 100, 300, 214);
    exitDoor = new Selection(btnExitDoor, rm, 50, this.Height - 150, 112, 75);
    exitDoor.Image("Exit_sign");
    leftSelection.Hide();
    rightSelection.Hide();
    exitDoor.Hide();

    imageBackground = Background.Museum_entrance;
    this.BackgroundImage = (Bitmap)rm.GetObject("Museum_entrance");
    soundPlayer.Stream = (Stream)rm.GetObject("Entrance_music");

    Say(1); // call the method that calls the bluetooth delegate to make
           // the bear say something

    try
    {
        // load characters into agent object
        mainAgent.Characters.Load("Peedy",
            @"C:\windows\msagent\chars\Peedy.acs");
    }

```

```

        // set current character to Peedy and show him
        agent = mainAgent.Characters["Peedy"];
    }
    catch (FileNotFoundException)
    {
        MessageBox.Show("Invalid characer location",
            "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }

    soundPlayer.Play();
    agent.Show(0); // display Peedy
}

private void Say(int fileNumber)
{
    returncode = 0;
    Thread.Sleep(threadSleep);
    tmrGameClock.Interval = 1;
    tmrGameClock.Enabled = true;
    IAsyncResult iftAR = b.BeginInvoke(ref _control, fileNumber, new
AsyncCallback(TalkComplete), fileNumber);
}

private void MyMuseum_Click(object sender, EventArgs e)
{
    // move the agent to the position that the user clicked
    try
    {
        agent.MoveTo(Convert.ToInt16(Cursor.Position.X - 60),
            Convert.ToInt16(Cursor.Position.Y - 60), 5);
    }
    catch {}

    switch (imageBackground) // check the current background
    {
        case Background.Museum_exit: // if the current background is museum exit
            Close(); // close the program
            break;
    }
}

```

```
    }  
    }  
  
    private void mainAgent_DblClick(object sender,  
AxAgentObjects._AgentEvents_DblClickEvent e)  
    {  
        switch (imageBackground)  
        {  
            case Background.Museum_entrance:  
                agent.Play("Acknowledge");  
                MuseumLobby();  
                break;  
            default:  
                agent.Play("Confused");  
                agent.Speak("Why are you poking me?", "");  
                agent.Play("RestPose");  
                break;  
        }  
    }  
}  
  
private void mainAgent_ClickEvent(object sender,  
AxAgentObjects._AgentEvents_ClickEvent e)  
{  
    switch (imageBackground)  
    {  
        case Background.Museum_entrance:  
            agent.Play("Acknowledge");  
            MuseumLobby();  
            break;  
        default:  
            agent.Play("Surprised");  
            agent.Play("RestPose");  
            break;  
    }  
}  
  
private void btnLeftSelection_Click(object sender, EventArgs e)  
{
```

```
        switch (imageBackground)
        {
            case Background.Museum_lobby:
                MuseumIdentify();
                break;

            case Background.Museum_identify:
                if (identifyGame.LeftName(round) ==
identifyGame.AnswerName(round))
                    Correct();
                else
                    Wrong();
                break;

            case Background.Museum_count:
                if (countGame.LeftName(round) == countGame.AnswerName(round))
                    Correct();
                else
                    Wrong();
                break;
        }
    }

    private void btnRightSelection_Click(object sender, EventArgs e)
    {
        switch (imageBackground)
        {
            case Background.Museum_lobby:
                MuseumCount();
                break;

            case Background.Museum_identify:
                if (identifyGame.RightName(round) ==
identifyGame.AnswerName(round))
                    Correct();
                else
                    Wrong();
                break;

            case Background.Museum_count:
                if (countGame.RightName(round) == countGame.AnswerName(round))
                    Correct();
                break;
        }
    }
}
```



```
        else
            Wrong();
        break;
    }
}

private void btnExitDoor_Click(object sender, EventArgs e)
{
    switch (imageBackground)
    {
        case Background.Museum_lobby:
            MuseumExit();
            break;
        case Background.Museum_identify:
            MuseumLobby();
            break;
        case Background.Museum_count:
            MuseumLobby();
            break;
    }
}

private void MuseumLobby()
{
    imageBackground = Background.Museum_lobby;
    CleanupBackground();
    soundPlayer.Stream = (Stream)rm.GetObject("Lobby_music");
    this.BackgroundImage = (Bitmap)rm.GetObject("Museum_lobby");
    soundPlayer.Play();
    exitDoor.Show();
    agent.Show(0);
    leftSelection.Image("Potato");
    rightSelection.Image("Book_worm");
    Say(2);
}

private void MuseumIdentify()
{

```

```
        imageBackground = Background.Museum_identify;
        CleanupBackground();
        soundPlayer.Stream = (Stream)rm.GetObject("Identify_music");
        this.BackgroundImage = (Bitmap)rm.GetObject("Museum_identify");
        soundPlayer.Play();
        exitDoor.Show();
        agent.Show(0);
        round = 0;
        identifyGame.NewGame();
        Say(7);
    }

    private void MuseumCount()
    {
        imageBackground = Background.Museum_count;
        CleanupBackground();
        soundPlayer.Stream = (Stream)rm.GetObject("Count_music");
        this.BackgroundImage = (Bitmap)rm.GetObject("Museum_count");
        soundPlayer.Play();
        exitDoor.Show();
        agent.Show(0);
        round = 0;
        countGame.NewGame();
        Say(8);
    }

    private void DisplayRound()
    {
        agent.Play("RestPose");

        leftSelection.Hide();
        rightSelection.Hide();

        btnLeftSelection.Enabled = false;
        btnRightSelection.Enabled = false;

        // show the left and right images
        switch (imageBackground)
```

```

    {
        case Background.Museum_identify:
            leftSelection.Image(identifyGame.LeftName(round));
            rightSelection.Image(identifyGame.RightName(round));
            break;

        case Background.Museum_count:
            leftSelection.Image(countGame.LeftName(round));
            rightSelection.Image(countGame.RightName(round));
            break;
    }

    leftSelection.Show();
    rightSelection.Show();

    Say(11);
}

private void MuseumExit()
{
    soundPlayer.Stop();
    agent.Play("Wave");
    CleanupBackground();
    this.BackgroundImage = (Bitmap)rm.GetObject("Museum_exit");
    soundPlayer.Stream = (Stream)rm.GetObject("Exit_music");
    tmrGameClock.Interval = 30000; // 30 seconds
    soundPlayer.Play();
    tmrGameClock.Enabled = true;
    imageBackground = Background.Museum_exit;
    Say(10);
}

private void CleanupBackground()
{
    agent.Hide(0);
    while (agent.Visible) { }
    agent.MoveTo(0, 0, 5);
    leftSelection.Hide();
    rightSelection.Hide();
}

```

```
        btnLeftSelection.Enabled = false;

        btnRightSelection.Enabled = false;

        exitDoor.Hide();

        soundPlayer.Stop();
    }

    private void tmrGameClock_Tick(object sender, EventArgs e)
    {
        switch (returncode)
        {
            case 0:
                switch (imageBackground)
                {
                    case Background.Museum_exit:
                        Close();
                        break;
                }
                break;

            case 1:
                returncode = 0;
                tmrGameClock.Enabled = false;
                switch (imageBackground)
                {
                    case Background.Museum_entrance:
                        agent.Play("Greet");
                        agent.Play("RestPose");
                        break;
                }
                break;

            case 2:
                returncode = 0;
                tmrGameClock.Enabled = false;
                switch (imageBackground)
                {
                    case Background.Museum_lobby:
                        Say(3);
                        leftSelection.Show();
                        break;
                }
            }
        }
    }
}
```

```
    }  
    break;  
case 3:  
    returncode = 0;  
    tmrGameClock.Enabled = false;  
    switch (imageBackground)  
    {  
        case Background.Museum_lobby:  
            Say(5);  
            rightSelection.Show();  
            break;  
    }  
    break;  
case 5:  
    returncode = 0;  
    tmrGameClock.Enabled = false;  
    switch (imageBackground)  
    {  
        case Background.Museum_lobby:  
            Say(6);  
            break;  
    }  
    break;  
case 7:  
    returncode = 0;  
    tmrGameClock.Enabled = false;  
    DisplayRound();  
    break;  
case 8:  
    returncode = 0;  
    tmrGameClock.Enabled = false;  
    DisplayRound();  
    break;  
case 11:  
    returncode = 0;  
    tmrGameClock.Enabled = false;  
    switch (imageBackground)  
    {
```

```
        case Background.Museum_identify:
            Say(identifyGame.AnswerNumber(round));
            break;
        case Background.Museum_count:
            Say(countGame.AnswerNumber(round));
            break;
    }
    break;
case 12:
    returncode = 0;
    tmrGameClock.Enabled = false;
    if (round < 2)
    {
        round++;
        DisplayRound();
    }
    else
    {
        round = 0;
        MuseumLobby();
    }
    break;
case 13:
    returncode = 0;
    tmrGameClock.Enabled = false;
    if (round < 2)
    {
        round++;
        DisplayRound();
    }
    else
    {
        round = 0;
        MuseumLobby();
    }
    break;
default:
    returncode = 0;
```

```
        tmrGameClock.Enabled = false;

        btnLeftSelection.Enabled = true;

        btnRightSelection.Enabled = true;

        break;
    }
}

private void Wrong()
{
    Answer();

    Say(13);

    agent.Play("Decline");
}

private void Correct()
{
    Answer();

    Say(12);

    agent.Play("Pleased");
}

private void Answer()
{
    switch (imageBackground)
    {
        case Background.Museum_identify:
            if (identifyGame.AnswerName(round) == identifyGame.LeftName(round))
                btnRightSelection.Enabled = false;
            else
                btnLeftSelection.Enabled = false;
            break;
        case Background.Museum_count:
            if (countGame.AnswerName(round) == countGame.LeftName(round))
                btnRightSelection.Enabled = false;
            else
                btnLeftSelection.Enabled = false;
            break;
    }
}
```


Code: Selection.cs

```
// Student Name: Oscar A. Smalls, Jr.
// Program Name: Selection.cs
// Date       : April 26, 2008

// Used to encapsulate the button and the image to be displayed on the button

using System;
using System.Collections.Generic;
using System.Text;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;
using System.Resources;
using System.Reflection;

namespace MyMuseum
{
    public class Selection
    {
        private Button selection;
        private ResourceManager rm;

        public Selection(Button select, ResourceManager resourceManager,
            int X, int Y, int width, int height)
        {
            selection = select;
            selection.Size = new Size(width, height);
            selection.Location = new Point(X, Y);
            rm = resourceManager;
            selection.Text = "";
        }

        public void Image(String resource)
        {
            selection.Image = (Bitmap)rm.GetObject(resource);
        }
    }
}
```

```
public void Hide()
{
    selection.Hide();
}

public void Show()
{
    selection.Show();
}
}
```

Chapter 7.

References

“555 and 556 Timer Circuits.” <<http://www.kpsec.freeuk.com/555timer.htm>>. 29

August 2007. 82 pars.

“97.497: RoboFlag – Course Resources – Technical Information – Infrared Robot Beacon.” <http://roboflag.carleton.ca/resources/technical/ir_beacon.shtml>. 28

August 2007. 70 pars.

“About the Propeller Chip.” <<http://www.parallax.com/tabid/433/Default.aspx>>. 24

April 2008.

“BASIC Stamp Syntax and Reference Manual”, Version 2.2, Parallax Inc., 2005

“Bluetooth Boe-Bot Robot Controlled by Microsoft Robotics Studio.”

<<http://www.parallax.com/dl/docs/prod/robo/MSRS-Bluetooth-Boe-Bot-v1.4.pdf>>. 01 November 2007.

Deitel and Deitel. C# for Programmers Second Edition. 2006.

“DLP-232PC Miniature USB – Microcontroller Module.”

<<http://www.dlpdesign.com/232pc-ds-v12.pdf>>. 29 August 2007.

“EmbeddedBlue eb500-SER User Manual.”

<<http://www.parallax.com/Portals/0/Downloads/docs/prod/comm/A7-UM-eb500-SER.pdf>>. 01 November 2007.

“IR Remote for the Boe-Bot”, Student Guide, Version 1.1, Parallax Inc., 2006

“Long Range Infrared Remote.”

<<http://www.boondog.com/tutorials/lrir/mc145026.htm>>. 27 August 2007.

“Panasonic PNA4611M Series datasheet.”

<http://eecs.oregonstate.edu/education/student_ref/active/ir_detectors/PNA4612M00XD.pdf>. 24 April 2008.

“Robot Room – Infrared Emitter from CMOS 555.”

<<http://robotroom.com/Infrared555.html>>. 28 August 2007.

“Robotics with the Boe-Bot”, Student Guide, Version 2.2, Parallax Inc., 2004

“Rogue Robotics uMP3 Playback Module Data Sheet.”

<<http://www.roguerobotics.com/files/ump3/documentation/UMP3-110-A1-102.pdf>>. 24 April 2008.

“Simple combination lock : DIGITAL INTEGRATED CIRCUITS.”

<http://www.allaboutcircuits.com/vol_6/chpt_7/7.html>. 16 October 2007.

“Understanding Signals”, Student Guide, Version 1.0, Parallax Inc., 2003

“What’s a Microcontroller?”, Student Guide, Version 2.1, Parallax Inc., 2003

Williams, Jon. “Sounds of the Season.” The Nuts and Volts of BASIC Stamps, Volume
6, California: Parallax, Inc., 2005